PROJECT GNOSIS

CODE ARCHAEOLOGY SYSTEM

OPERATIONS HANDBOOK · SYSTEMS DATA

This handbook describes the Project Gnosis code-archaeology system as it exists in the working tree at commit 4135A25, dated 16 October 2026. It is written for the engineer who must operate, extend or review the system without prior knowledge of it.

The system accepts a public GitHub repository address and returns a grounded architectural document: a file manifest, symbol tables, an import graph, per-file complexity scores, a vector index of semantic code chunks, and natural-language explanations produced by a hosted language model. Seven deterministic stages surround one non-deterministic stage. A shared state object carries the work between them.

Every statement is classified. **OBSERVED** means read from executable source in this tree. **DOCUMENTED** means asserted by a repository document or commit and not independently exercised. **INFERRED** means a conclusion drawn from structure, with its basis named. Where a document and the code disagree, the code is described and the disagreement is recorded.

DOCUMENT	GN1A-03-BLOCK I-(1), Revision 1	BASIC DATE	25 July 2026
PROGRAM	Project Gnosis / Archaeon	CHANGE DATE	16 October 2026
BASIS	139 commits; four uncommitted application paths	PAGES	1-1 through 8.2-1
PREPARED BY	Handbook working group, from source inspection	LICENSE	AGPL-3.0 or commercial

Contents

1	General: scope, reading map, purpose, architecture	1-1 – 1-4
2	Computing equipment and software: declared and resolved stack	2.1-1 – 2.1-2
3	Subsystems 1–9: agents, API, store, queue, display	3.1-1 – 3.9-1
4	Operation: end-to-end flow, state, errors, retries	4.1-1 – 4.2-1
5	Equipment list: every repository-owned file	5.1-1 – 5.2-1
6	Design record: decisions, tradeoffs, documentation drift	6.1-1 – 6.2-1
7	Checkout: starting the system and running the tests	7.1-1 – 7.2-1
8	Appendix: defects, limitations, source verification, open items	8.1-1 – 8.2-1

NOTICE

This document is generated from a local working tree. It does not certify any deployed environment, credential, cloud setting or live service. Secret values are neither reproduced nor inferred. Figures are simplified; the source-verification appendix names the code behind each claim.

SECTION 1 · SUBSECTION 1.1
SCOPE AND READING MAP

1.1.1 INTRODUCTION.

This handbook is the systems-data volume for the Project Gnosis code-archaeology system. It describes what the system is, how its nine subsystems work, the order in which they execute, every file that belongs to the repository, and the decisions that shaped them. The single idea the reader must leave with is that Gnosis is a deterministic seven-stage compiler wrapped around one hosted language-model call, not an autonomous agent swarm (figure 1.1-1).

The document is organised for three readers. An operator starts at section 7 and uses section 3 for fault isolation. A reviewer reads sections 1, 4 and 6. A maintainer reads section 3 in pipeline order, then section 5 for the file inventory. Section 8 records defects and unresolved questions so that no reader mistakes a known gap for an undocumented feature.

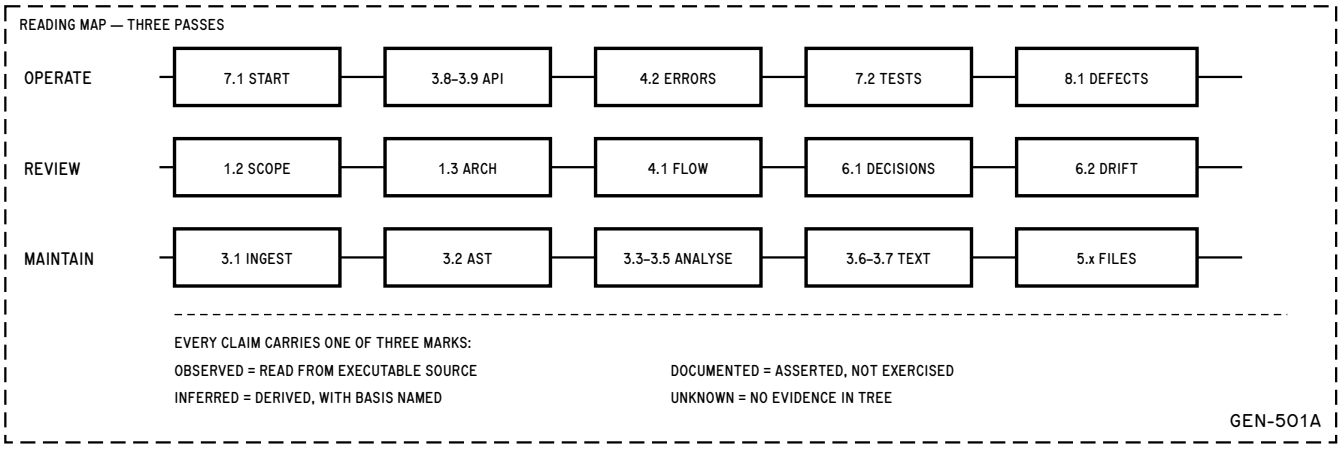


Figure 1.1-1. Reading Map and Evidence Marks

1.1.2 HOW TO USE THIS DOCUMENT.

The page number is the address. A reference such as 3.5-1 means section 3, subsection 3.5, sheet 1. Figure and table numbers run continuously inside a subsection. Paragraph numbers are stable and are used in the source-verification appendix.

Three marks qualify evidence. **OBSERVED** statements were read from source, configuration or test code in this working tree. **DOCUMENTED** statements come from a repository prose file, comment or commit message and were not exercised. **INFERRED** statements are conclusions; the basis is named in the sentence. A claim that cannot be placed in one of these classes is listed as an open item in 8.2-1.

1.1.3 SECTION INDEX.

Table 1.1-1. What Each Section Answers

Section	Question answered
1.2	What problem is solved, and what is explicitly outside the boundary?
1.3	What are the parts, and how are they deployed?
2.1	Which software is declared, which version actually resolves, and what does it do?
3.1–3.7	How does each pipeline stage transform state?
3.8–3.9	How is work submitted, stored, queued and displayed?
4.1–4.2	In what order does execution occur, and what happens on failure?
5.1–5.2	What is every file for, and what is deliberately excluded?
6.1–6.2	Why was each choice made, and where do documents disagree with code?
7.1–7.2	How is the system started, and what has actually been tested?
8.1–8.2	What is broken, limited, simplified or still unknown?

1.1.4 Reading Sequences.

Table 1.1-2. Suggested Reading Sequences

Reader	Order
Newcomer	1.1 → 1.2 → 1.3 → 4.1 → 2.1-1
Maintainer	3.1 → 3.2 → 3.3–3.5 → 3.6–3.9 → 4.1 → 5.1-1
Reviewer	1.3 → 6.1 → 6.2 → 8.1 → 8.2
Operator	7.1 → 3.8-1 → 4.2 → 7.2 → 8.1
Verifier	8.2 → cited file → test → verdict

CAUTION

Repository prose is stale in several places. The README still describes a sequential pipeline, React 18, an in-memory-only job store, an *azure-deploy.yml* workflow and 407 tests. The code and the test run show otherwise. Read 6.2-1 before quoting any document as fact.

FIRST PASS	ANY READER

```
1 1.2 - read the boundary
2 1.3 - trace figure 1.3-1
3 4.1 - follow one job
  √ STATE - seven stages, one store
4 3.x - open the stage in question
5 6.2 - check the drift ledger
6 8.1 - read known defects
```

GEN

SECTION 1 · SUBSECTION 1.2
SYSTEM PURPOSE AND BOUNDARIES

1.2.1 INTRODUCTION.

Project Gnosis converts an unfamiliar public GitHub repository into a reading aid. A user submits a repository address; the system fetches a filtered copy of its text files, parses them into symbols, measures their structural weight, and returns a document that names the entry points, the risky files and a safe reading order. The system never executes the target repository (figure 1.2-1).

The boundary is deliberately narrow. Gnosis reads source text and repository metadata. It does not build, run, test, patch or deploy the analysed project, and it cannot see a private repository unless the caller supplies a token through the API. Its output is a snapshot of one branch at one commit; it does not update when the repository changes.

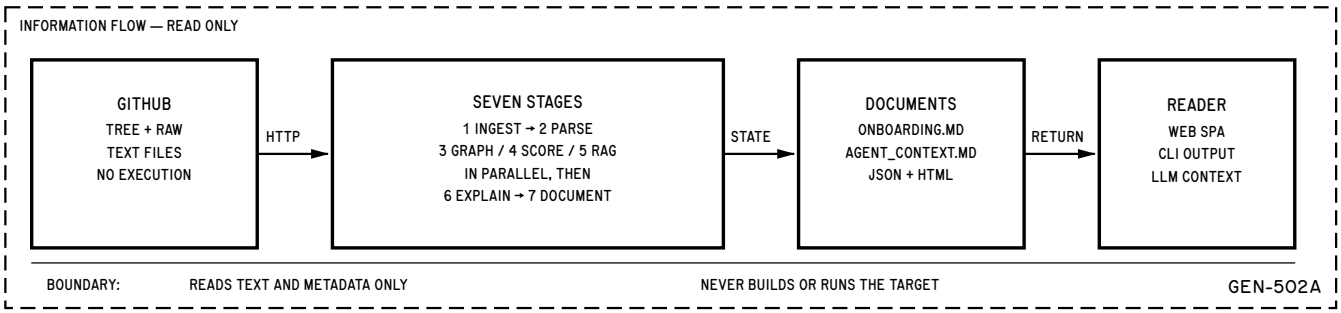


Figure 1.2-1. Input, Seven-Stage Transformation and Output

1.2.2 FUNCTIONAL DESCRIPTION.

Stage 1 lists every path on the default branch, keeps recognised source and configuration extensions, then fetches the raw text of each file. Stage 2 parses each file with a tree-sitter grammar and records functions, classes, imports and docstrings. Stages 3, 4 and 5 run concurrently: the graph resolves internal imports, the scorer computes cyclomatic complexity, and the retrieval stage chunks and embeds the code. Stage 6 selects the most important files and asks a hosted model to explain them; stage 7 assembles the results into Markdown and JSON (figure 1.2-2).

1.2.2.1 Repository Sizing.

A repository is classified by the number of files that survive filtering. At most 300 files is FULL; up to 1,000 is FULL (WARNING); up to 3,000 is SAMPLED. The API rejects anything above 3,000 before a job is created; the CLI caps the manifest at 3,000 and continues. Sorting is by path depth, then alphabetically, so shallow files survive a cap first.

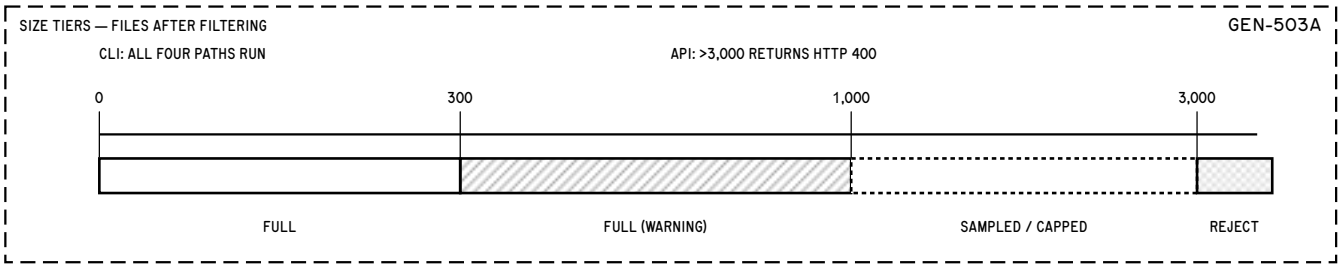


Figure 1.2-2. Repository Size Tiers and Enforcement Point (band widths not to scale)

1.2.3 PERFORMANCE AND DESIGN DATA.

Table 1.2-1. Boundary and Sizing Parameters

Parameter	Value
Full-analysis ceiling	300 files (env MAX_FULL_ANALYSIS_FILES)
Warning ceiling	1,000 files (MAX_WARNING_ANALYSIS_FILES)
Sampled / ingestion cap	3,000 files (MAX_SAMPLED_ANALYSIS_FILES)
Per-file size ceiling	150,000 bytes
Accepted extensions	18, from .py to .md
Explanation cap	20 files default; 0–100 allowed
Output languages	8 parsed; 8 scored
Repository write access	None

CAUTION

A sampled run is not a whole-repository verdict. Only the first 3,000 filtered paths, sorted by depth, are read. Files that are not ingested cannot appear in the graph, the scores or the document. The header records the mode; the reader must not generalise from a sample.

SCOPE CHECK	REVIEWER
1 CONFIRM - public URL, one branch	
2 CONFIRM - text only, no target build	
3 CONFIRM - no private repo without token	
√ MODE - Full, Warning or Sampled	
4 CONFIRM - output is a snapshot	

SECTION 1 · SUBSECTION 1.3
ARCHITECTURE AND DEPLOYMENT

1.3.1 INTRODUCTION.

The system has one analysis engine and two delivery paths. The engine is a Python package under *src/* whose seven stages communicate only through an *ArchaeonState* dataclass. The delivery paths are a command-line runner, *run.py*, and a FastAPI service that stores job state and either dispatches to a Redis queue or runs the pipeline in a local thread. The single idea the reader must leave with is that the API layer does not contain analysis logic; it contains scheduling and status (figure 1.3-1).

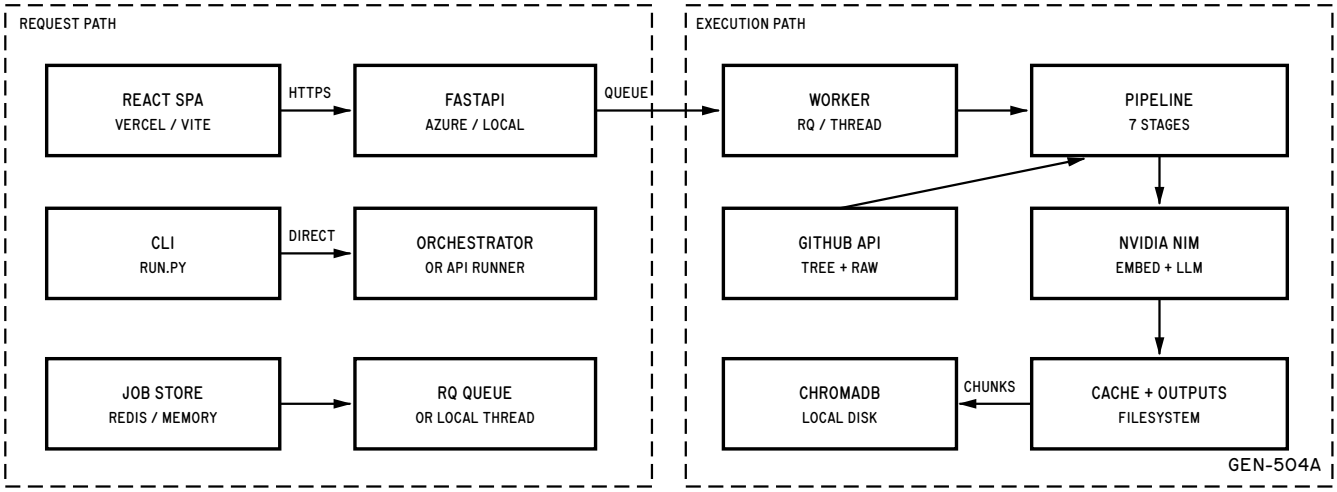


Figure 1.3-1. Runtime Components and Two Delivery Paths

1.3.2 FUNCTIONAL DESCRIPTION.

The API validates the URL and repository size, creates a job, then enqueues it when *REDIS_URL* is reachable or submits a local thread otherwise. Either runner constructs an *ArchaeonState*, calls the seven stages in sequence (fanning stages 3–5 out to three threads) and writes a status transition at each boundary. The SPA polls the job endpoint and renders the stored result; the CLI prints progress and writes files under *outputs/*.

1.3.2.1 State as the Only Contract.

No stage imports another stage. Each exposes *run(state) -> state*, reads named fields and writes named fields. The orchestrator is the only caller. This is what makes a stage testable in isolation and what allows stages 3, 4 and 5 to run concurrently without locks: they write disjoint fields.

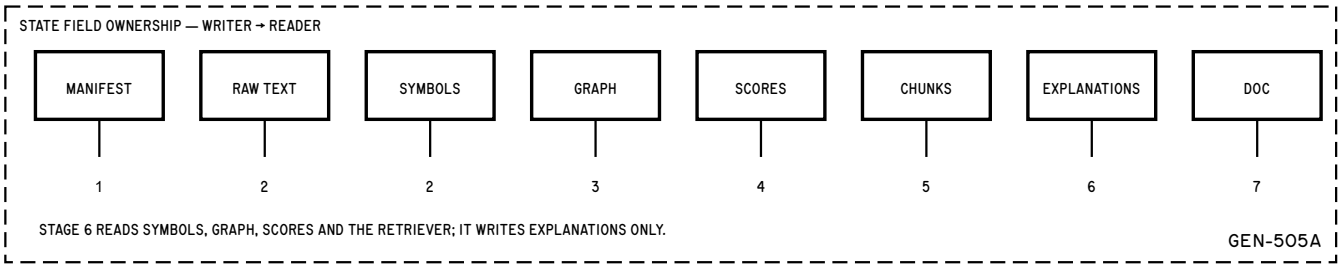


Figure 1.3-2. State Field Ownership by Stage

1.3.3 PERFORMANCE AND DESIGN DATA.

Table 1.3-1. Runtime Components

Component	Implementation	Responsibility
Display	React SPA	Submit, poll, render results
API	FastAPI + Uvicorn	Validate, deduplicate, schedule, report
Job store	Redis or dict	Status, result, active index
Queue	RQ or thread	Decouple submission from execution
Worker	Python module	Consume jobs, serve health on PORT
Engine	7 Python stages	Transform repository text to documents
Vector store	ChromaDB 1.5.9	Per-job chunk collection
External	GitHub, NVIDIA	Source text, embeddings, explanations

CAUTION

The local-thread path and the Redis path share *pipeline_runner.run*. If Redis is configured but the worker is not running, jobs remain queued indefinitely; the API does not detect a missing consumer. Confirm the worker before diagnosing a stuck job.

PATH SELECTION	OPERATOR
1 READ - REDIS_URL in environment	
2 PROBE - redis reachable	
√ SET - queue gnosis_jobs	
3 START - python -m src.api.worker	
4 VERIFY - job leaves queued	

GEN

SECTION 2 · SUBSECTION 2.1
COMPUTING EQUIPMENT AND SOFTWARE

2.1.1 INTRODUCTION.

The backend is a Python 3 application with no framework beyond FastAPI at its edge. Analysis depends on five libraries that do the substantive work: tree-sitter for syntax, NetworkX for graph algorithms, Radon for Python complexity, ChromaDB for vector storage, and the OpenAI client pointed at NVIDIA NIM. The one idea the reader must leave with is that *requirements.txt* declares floors only; the versions in this handbook are the ones resolved in the local virtual environment, not a locked contract (figure 2.1-1).

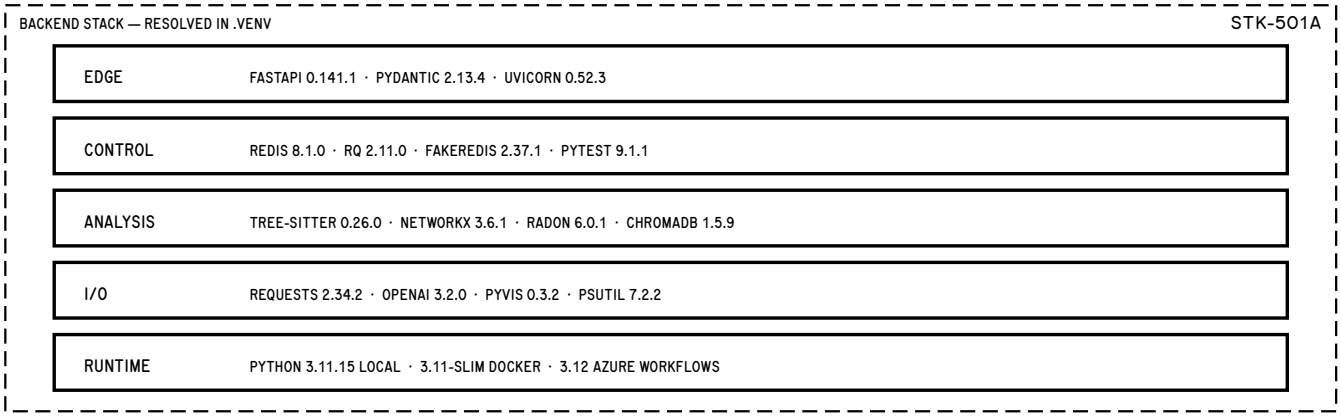


Figure 2.1-1. Backend Dependency Layers and Resolved Versions

2.1.2 FUNCTIONAL DESCRIPTION.

FastAPI parses and validates requests with Pydantic models, exposes the routes, and delegates every long operation to a job runner. Requests and the OpenAI client carry out HTTPS calls to GitHub and NVIDIA. Tree-sitter supplies a concrete syntax tree per language; the parsers in *src/parsers/* convert those trees to symbol tables. NetworkX holds the file graph in memory and supplies cycle detection, topological sorting and PageRank. Radon computes Python cyclomatic complexity; the JavaScript and generic paths use a custom branch counter. ChromaDB persists chunk embeddings under *./chroma_db*, and RQ moves job identifiers through Redis.

2.1.2.1 SQLite Substitution.

At the top of *src/api/main.py*, before any ChromaDB import, the module imports *pysqlite3* and replaces *sys.modules["sqlite3"]* with it. The comment states that Azure App Service ships SQLite 3.31 while ChromaDB requires at least 3.35. The substitution is unconditional, so it also runs locally; it is an environment workaround, not an analysis feature.

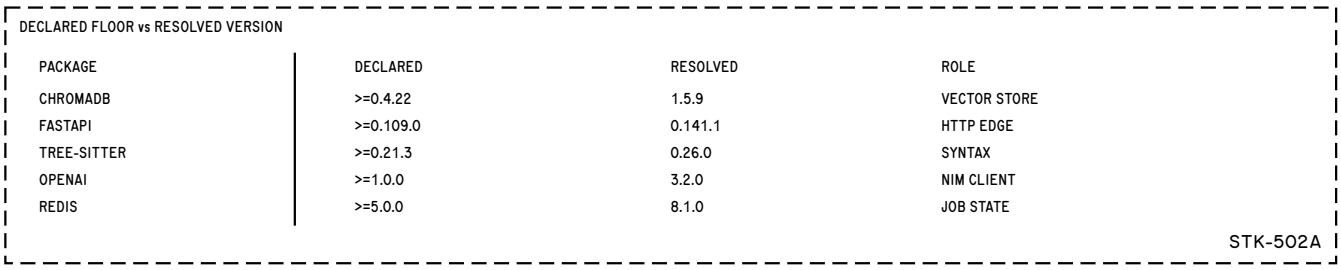


Figure 2.1-2. Declaration Floors and Local Resolution

2.1.3 PERFORMANCE AND DESIGN DATA.

Table 2.1-1. Backend Software and Actual Role

Package	Resolved	Role in this system
chromadb	1.5.9	Per-job collection of embedded code chunks
networkx	3.6.1	DiGraph, simple_cycles, topological_sort, pagerank
radon	6.0.1	Python cyclomatic complexity and line metrics
tree-sitter	0.26.0	Parser runtime for eight language grammars
fastapi	0.141.1	Routes, validation, middleware
pydantic	2.13.4	Request and response schemas
openai	3.2.0	OpenAI-compatible client for NVIDIA NIM
redis / rq	8.1.0 / 2.11.0	Job records and distributed queue
pytest	9.1.1	476-test verification suite

CAUTION

There is no Python lockfile. Every requirement is a lower bound, so two hosts can resolve different versions. The frontend does have a lockfile. Reproduce the versions in table 2.1-1 before attributing a defect to the code.

```
STACK CHECK                          MAINTAINER
1 READ - requirements.txt floors
2 RUN - python -m pip list
  √ MATCH - table 2.1-1
3 CHECK - Python 3.11 or 3.12
4 VERIFY - sqlite3 module is pysqlite3
```

SECTION 2 · SUBSECTION 2.1 (CONT.)
COMPUTING EQUIPMENT AND SOFTWARE

2.1.4 INTRODUCTION.

The operator display is a single-page React application built by Vite and served as static files. It has one source file of consequence, *App.tsx*, holding the landing, progress, results and error screens plus an inline design system. The point to retain: navigation is local component state, not a router — *react-router-dom* is installed but never imported by the entry point (figure 2.1-3).

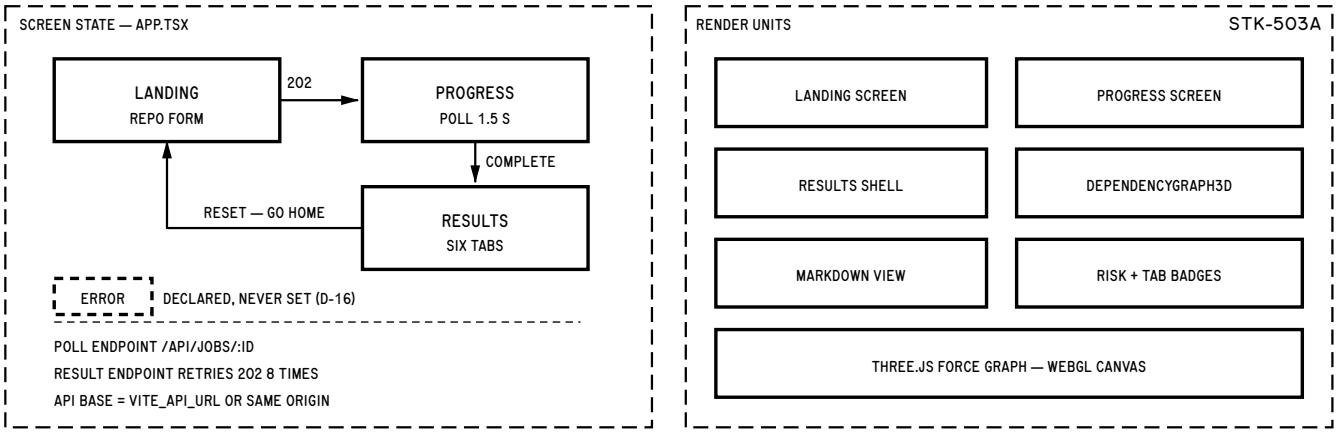


Figure 2.1-3. Screen State and Rendering Units

2.1.5 FUNCTIONAL DESCRIPTION.

main.tsx mounts the application inside a Sentry error boundary and a PostHog provider, then appends Vercel Analytics and Speed Insights. *App.tsx* submits *{repo_url, max_explanations, skip_llm}* to */api/analyze*, stores the returned job identifier, and polls */api/jobs/:id* every 1,500 milliseconds. When status becomes complete, it fetches */api/jobs/:id/result* and renders six tabs: onboarding, agent context, explanations, dependency graph, complexity and raw JSON. The graph tab hands the full node and link arrays to a WebGL force layout. A reset returns the operator to the landing form; the error screen is declared and rendered but never set, so failures surface in the progress screen (defect D-16).

2.1.5.1 Build and Headers.

Vite builds static assets into *dist/*. Vercel serves them with a Content-Security-Policy and security headers from *frontend/vercel.json*, and proxies PostHog under */ingest* and */decide*. The same file rewrites unknown paths to *index.html* — conventional for a single-page application, unnecessary while the router is unused.

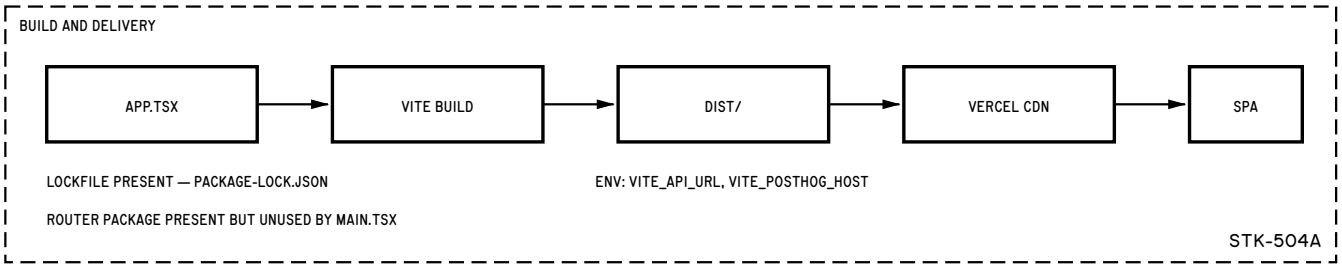


Figure 2.1-4. Build Chain and Delivery Surface

2.1.6 PERFORMANCE AND DESIGN DATA.

Table 2.1-2. Frontend Software, Declared and Resolved

Package	Declared	Resolved	Role
react / react-dom	^19.0.0	19.2.8	UI runtime
vite	^8.0.0	8.2.0	Build and dev server
typescript	^5.7.0	5.9.3	Type checking
tailwindcss	^4.0.0	4.3.3	Utility CSS
3d-force-graph	^1.80.1	1.80.1	Graph layout and camera
three	^0.179.1	0.179.1	WebGL renderer
motion	^12.0.0	12.43.0	Screen transitions
react-markdown	^10.1.0	10.1.0	Document rendering
react-router-dom	^7.18.2	7.18.2	Installed, not imported
@sentry/react	^10.73.0	10.73.0	Error reporting

CAUTION

main.tsx contains fallback Sentry and PostHog identifiers. They are not reproduced here. Treat them as public client configuration, rotate them through environment variables, and do not add server secrets to any *VITE_* variable; Vite inlines those into the bundle.

DISPLAY CHECK **OPERATOR**

```
1 SET - VITE_API_URL
2 RUN - npm run dev
3 OPEN - localhost:5173
  √ SUBMIT - job id returns
4 WATCH - progress advances
5 VERIFY - six tabs render
```

STK

SECTION 3 · SUBSECTION 3.1
FILE INGESTION

3.1.1 INTRODUCTION.

Ingestion is the only stage that talks to GitHub. It obtains the complete path list for one branch, keeps the files that are worth reading, classifies the repository by size, and downloads the raw text of each kept file. It writes two large fields: a manifest of file metadata and a dictionary of path-to-text. The one idea the reader must leave with is that the manifest, not the repository, defines everything downstream; a file absent here cannot exist for later stages (figure 3.1-1).

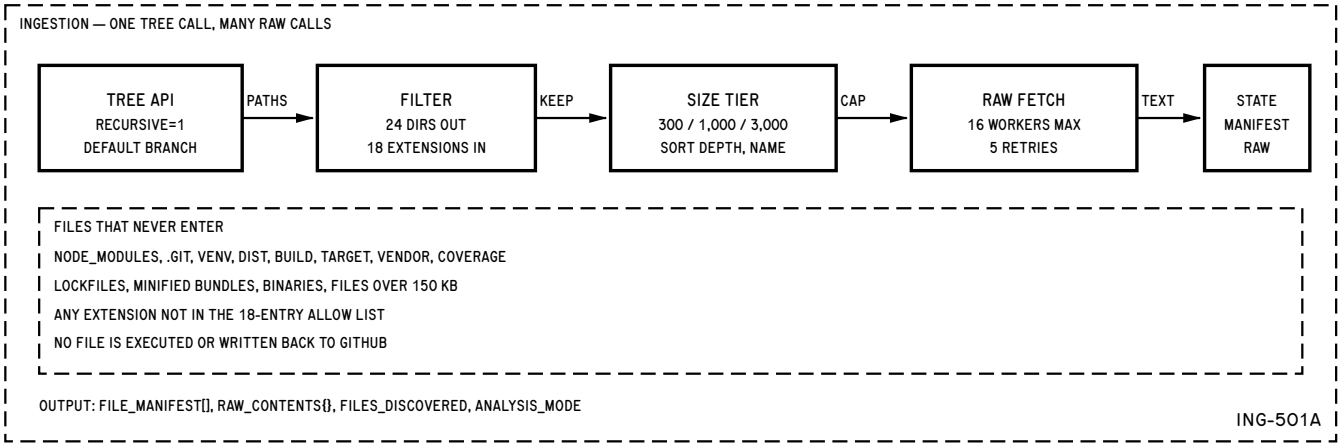


Figure 3.1-1. Ingestion Flow and Exclusion Boundary

3.1.2 FUNCTIONAL DESCRIPTION.

The stage calls *fetch_file_tree* once. For each entry it applies *should_include_file*, which checks directory segments first, then exact filenames, then minified suffixes, then the extension allow list, then the 150 KB ceiling. Surviving entries are sorted by slash count and then by path, so shallow files are preferred when a cap applies. The stage sets *analysis_mode* and calls *fetch_file_contents_batch*, which uses a shared requests session, a thread pool capped at sixteen, and five attempts with backoff. Line counts are computed from the returned text rather than trusted from the API.

3.1.2.1 Incremental Manifest Diff.

When *state.is_incremental* and a previous state exists, *compute_manifest_diff* classifies paths as added, modified, deleted or unchanged by SHA. Only changed paths are fetched; unchanged raw text is copied from the previous state when available. A deleted file is therefore visible to later stages as a diff entry, not as content.

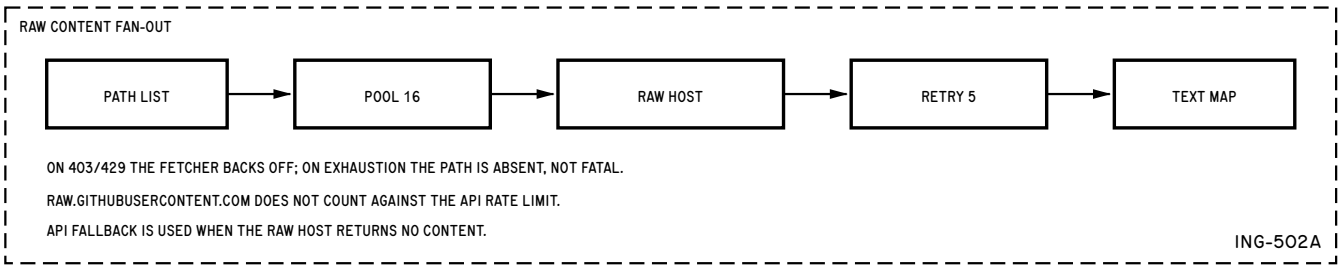


Figure 3.1-2. Content Fetch Pool and Retry Path

3.1.3 PERFORMANCE AND DESIGN DATA.

Table 3.1-1. Ingestion Parameters

Parameter	Value
Tree requests per run	1
Content workers	min(16, paths)
Fetch attempts	5, 0.5 s backoff
Inter-request delay	0.05 s
Maximum file size	150,000 bytes
Excluded directories	24 named patterns
Accepted extensions	18
Sort key	(depth, path)

CAUTION

The README says fifteen concurrent threads; the code caps at sixteen. It also lists a *tests/fixtures/sample_repo/* directory that is empty and untracked here. Trust table 3.1-1 and section 5.1-2, not the README tree.

INGESTION CHECK	OPERATOR
1 CONFIRM - GITHUB_TOKEN set	
2 READ - files discovered	
√ TIER - mode matches count	
3 CHECK - manifest not empty	
4 VERIFY - raw text for sample path	

ING

SECTION 3 · SUBSECTION 3.2
SYMBOL EXTRACTION

3.2.1 INTRODUCTION.

Symbol extraction turns text into a language-neutral inventory. Each file is parsed by a tree-sitter grammar; three extractor families then walk the tree and emit functions, classes and imports into a *SymbolTable*. The one idea the reader must leave with is that later stages never see syntax trees. They see only the flat records defined in *src/parsers/base.py* (figure 3.2-1).

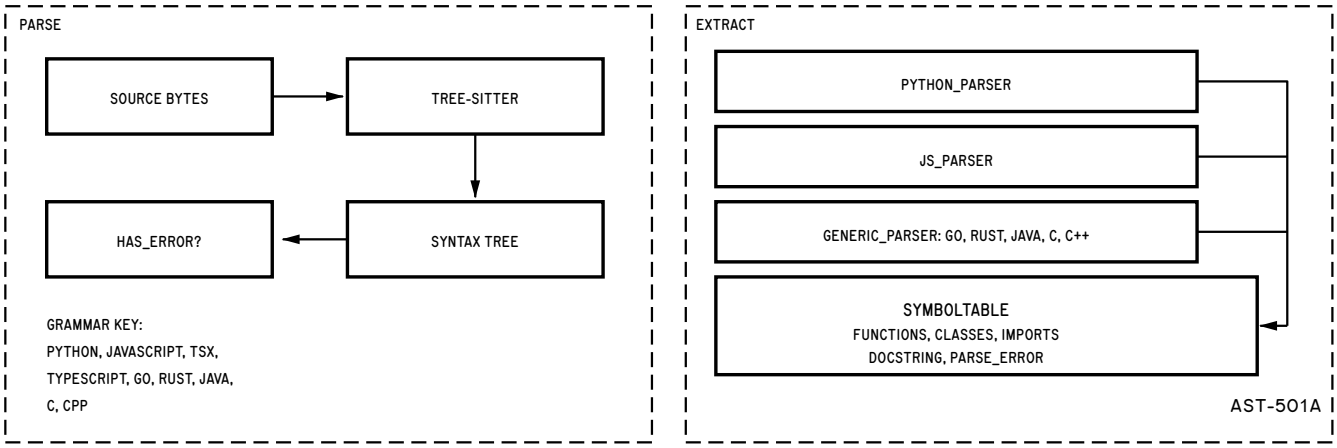


Figure 3.2-1. Parse and Extract Paths for Eight Languages

3.2.2 FUNCTIONAL DESCRIPTION.

run builds the set of manifest paths, selects files to parse, and submits them to a thread pool sized *min(12, CPU count, file count)*. Each worker calls *_parse_single_file*, which selects a grammar key, parses bytes, and dispatches to *extract_python*, *extract_js* or *extract_generic*. Python methods are recorded both as functions and as class method names; JavaScript arrow functions and exported declarations are recognised; the generic extractor handles receiver-based Go methods, Rust impl blocks, Java constructors and C declarators. After parsing, *_resolve_internal_imports* marks each import as internal or external against the manifest.

3.2.2.1 Error Tolerance.

A tree-sitter syntax error does not discard the file. The extractor still emits whatever symbols it can and sets *parse_error=True* with a detail string. An exception during extraction produces an empty table flagged as an error. Downstream, the file is skipped by retrieval and forced to CRITICAL risk, so a parse failure is visible rather than silent.

```
# src/agents/ast_parser.py, _parse_single_file (condensed)
grammar_key = "TSX" if path.endswith(".tsx") else lang
parser = get_parser(grammar_key)          # None -> skip, not error
tree = parser.parse(source_bytes)
has_error = tree.root_node.has_error      # partial tree still usable
docstring, functions, classes, imports = extract_python(tree, source_bytes)
st = SymbolTable(file_path=path, language=lang, module_docstring=docstring,
                 functions=functions, classes=classes, imports=imports,
                 parse_error=has_error)   # error is data, not an exception
```

3.2.3 PERFORMANCE AND DESIGN DATA.

Table 3.2-1. Grammars and Symbol Coverage

Language	Package	Coverage
Python	tree-sitter-python	functions, async, classes, methods, imports
JavaScript	tree-sitter-javascript	declarations, arrows, classes, imports
TypeScript	tree-sitter-typescript	TypeScript and TSX grammars
Go	tree-sitter-go	functions, methods, structs, imports
Rust	tree-sitter-rust	functions, impls, use declarations
Java	tree-sitter-java	classes, methods, imports
C / C++	tree-sitter-c / cpp	functions, structs, includes
Config / docs	none	skipped after ingestion

CAUTION

Symbol coverage is not semantic type resolution. A dynamic import, a generated macro, or a name assembled at run time will not appear. The absence of a symbol in table 3.2-1's output is not evidence that the symbol does not exist in the target repository.

PARSE CHECK	MAINTAINER
1 READ - symbol_tables count	
2 COMPARE - manifest count	
√ PARSE_ERROR - false for sample	
3 OPEN - one function record	
4 CONFIRM - line range matches file	

AST

SECTION 3 · SUBSECTION 3.3
DEPENDENCY GRAPH

3.3.1 INTRODUCTION.

The dependency stage converts internal imports into a directed graph whose nodes are manifest files and whose edges point from importer to imported file. It then computes in-degree, out-degree, PageRank, membership in a cycle, and, when the graph is acyclic, a topological order that becomes the suggested reading sequence. The one idea the reader must leave with is the edge direction: an edge A → B means A imports B, so high in-degree marks a file that many others depend on (figure 3.3-1).

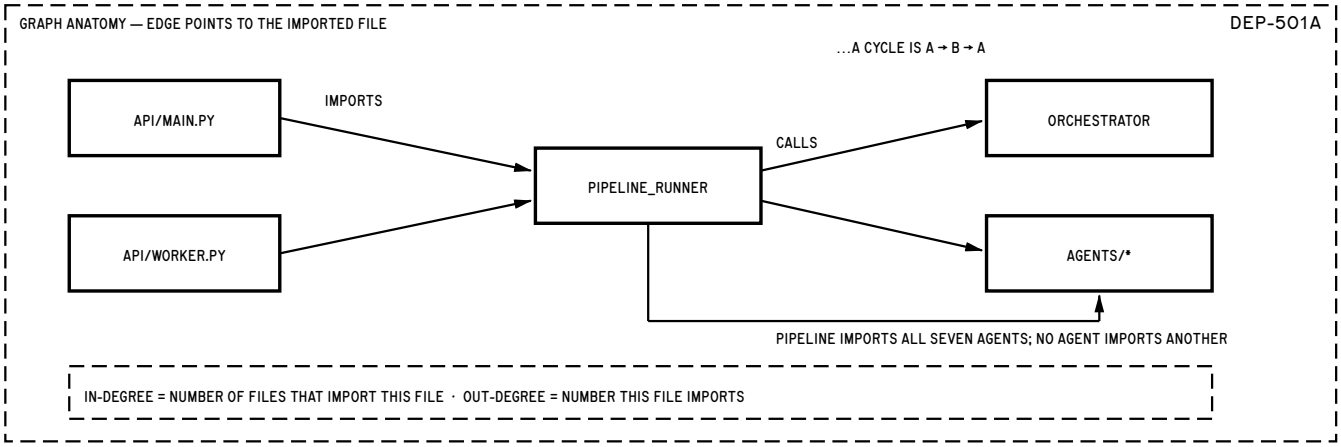


Figure 3.3-1. Edge Direction and Degree Definitions

3.3.2 FUNCTIONAL DESCRIPTION.

run adds every manifest file as a node before adding any edge, so isolated files remain visible to scoring and explanation. For each internal import it calls *resolve_import_to_paths*, which applies language-specific rules: Python absolute and relative module resolution, JavaScript extension and index resolution, Rust crate and *super* paths, C include paths, Go package directories, and Java source roots. Self-loops and targets outside the manifest are discarded. The stage then runs *nx.simple_cycles*, marks every node in a cycle, computes PageRank, and builds one statistics record per node.

3.3.2.1 Reading Order.

If the graph has no cycle, *nx.topological_sort* produces an order in which every dependency appears before its importer. If any cycle exists, the stage leaves the topological order empty rather than returning a partial order; the document then falls back to safe starting files. This is a deliberate all-or-nothing rule, not an oversight.

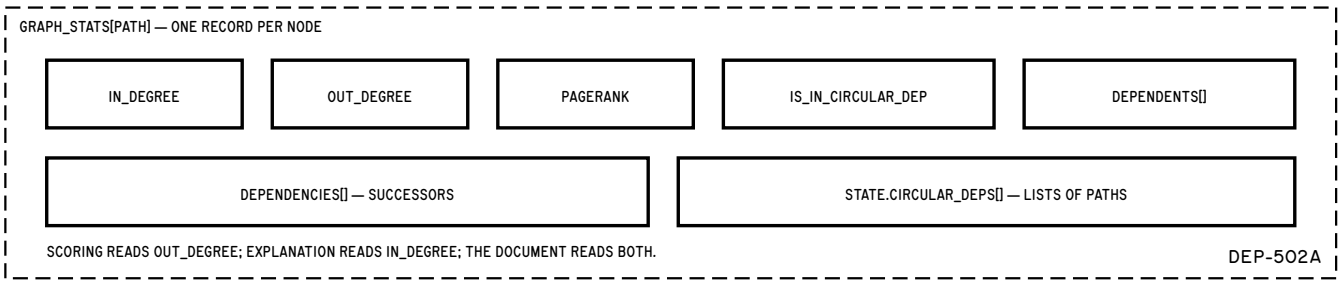


Figure 3.3-2. Statistics Record and Its Consumers

3.3.3 PERFORMANCE AND DESIGN DATA.

Table 3.3-1. Graph Construction Rules

Rule	Value
Node set	every manifest file
Edge	importer → imported
External imports	no edge
Self-loops	discarded
Missing targets	counted unresolved, dropped
Cycle algorithm	nx.simple_cycles
Rank	PageRank, 6 decimals
Topological order	empty if any cycle

CAUTION

simple_cycles enumerates every elementary cycle, and the cost can grow sharply on dense graphs. A repository with many mutually importing packages may spend most of stage 3 in cycle enumeration. The count is exact; it is not a sample.

```
GRAPH CHECK                          MAINTAINER
1 READ - node count equals manifest
2 PICK - one known import
  √ EDGE - direction is importer → target
3 CHECK - cycle list non-empty only if real
4 CONFIRM - topo empty when cycles exist
```

SECTION 3 · SUBSECTION 3.4
COMPLEXITY SCORING

3.4.1 INTRODUCTION.

Complexity scoring assigns each source file one of four risk levels from independent structural signals: syntax failure, cycle membership, maximum function complexity, average complexity, coupling, undocumented ratio and line count. The one idea the reader must leave with is that the signals combine by OR, not AND. A file reaches the highest level any single signal justifies (figure 3.4-1).

RISK LADDER — HIGHEST REACHED SIGNAL WINS

SIGNAL	MEDIUM AT	HIGH AT	CRITICAL AT
MAX FUNCTION CC	—	11	21
AVERAGE CC	6	10	—
COUPLING (OUT)	4	8	—
UNDOCUMENTED RATIO	0.60	0.80	—
LINE COUNT	300	600	—
PARSE ERROR	—	—	YES
IN CYCLE	—	—	YES

A DASH MEANS THE SIGNAL CANNOT PRODUCE THAT LEVEL ON ITS OWN.
LOW = EVERY SIGNAL BELOW ITS MEDIUM COLUMN; ANY ONE SIGNAL RAISES THE FILE.
UNDOCUMENTED RATIO IS IGNORED BELOW 3 FUNCTIONS PER FILE.

CPX-501A

Figure 3.4-1. Risk Signal Thresholds

3.4.2 FUNCTIONAL DESCRIPTION.

For Python, Radon's *cc_visit* returns per-function complexity; for JavaScript, TypeScript and the generic languages, the scorer walks the syntax tree and counts branch node types (if, loop, catch, ternary, logical operators, switch cases). Files are scored in a thread pool of up to twelve workers. Because coupling comes from the graph, which runs concurrently, the scorer first writes a provisional coupling count from symbol-table imports. *apply_graph_coupling* then replaces it with the graph out-degree, updates cycle membership, and recomputes the verdict for every file.

3.4.2.1 Verdict Rule.

The evaluator checks CRITICAL signals first and returns immediately when one fires. Otherwise it checks each HIGH signal, then each MEDIUM signal. Reasons accumulate, so a file can carry several explanations for one level. A clean file with no function and no signal is LOW with an empty reason list.

```
# src/agents/complexity_scorer.py, _compute_risk (condensed)
if parse_error:           reasons.append("File contains syntax errors"); level = "CRITICAL"
if is_in_circular_dep:    reasons.append("Involved in a circular dependency"); level = "CRITICAL"
if max_complexity >= 21:  reasons.append(f"Function '{_trunc(str(max_complexity))}' ..."); level = "CRITICAL"
if level == "CRITICAL":   return level, reasons        # short-circuit
if max_complexity >= 11:  level = "HIGH"               # then avg, coupling, docs, lines
# note: the CRITICAL reason interpolates the score where the function name belongs
```

3.4.3 PERFORMANCE AND DESIGN DATA.

Table 3.4-1. Risk Thresholds and Fields

Signal	Medium	High	Critical
Max function CC	—	11	21
Average CC	6.0	10.0	—
Coupling (out-degree)	4	8	—
Undocumented ratio	0.60	0.80	—
Line count	300	600	—
Parse error	—	—	yes
In cycle	—	—	yes
Undoc. gate (functions)	≥3	≥3	—

CAUTION

These are heuristics, not defects. A generated parser with a 25-branch switch is CRITICAL by rule even if it is correct. Use the level to choose what to read first, not as a quality judgement. The recorded reason text for CRITICAL max-CC is malformed; see 8.1-1.

SCORE CHECK	MAINTAINER
1 PICK - a small clean file	
2 CONFIRM - LOW, no reasons	
√ CC - matches manual count	
3 CHECK - coupling equals graph out-degree	
4 VERIFY - cycle file is CRITICAL	

3.4.4 WORKED READING.

A 720-line file with average complexity 4.0 and a worst function at 23 is CRITICAL, not HIGH: the evaluator returns on the critical signal before it tests line count. A worst function at 12 in the same file is HIGH twice over — the complexity and line-count reasons are both kept.

SECTION 3 · SUBSECTION 3.5
CODE RETRIEVAL

3.5.1 INTRODUCTION.

The retrieval stage builds a semantic index of the repository. It does not split text every N tokens; it emits one chunk for the module header, one for each function and one for each class, preserving code boundaries. Chunks are embedded by a hosted NVIDIA model and written to a ChromaDB collection scoped to the job. The one idea the reader must leave with is that stage 6 never reads whole files from GitHub; it reads these chunks (figure 3.5-1).

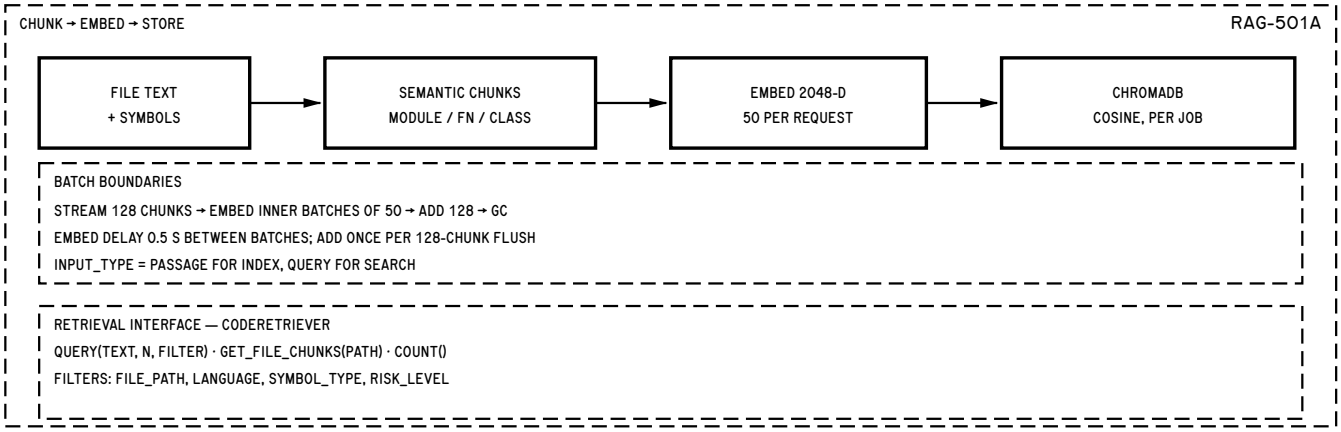


Figure 3.5-1. Semantic Chunking, Embedding and Retrieval Interface

3.5.2 FUNCTIONAL DESCRIPTION.

run names the collection *gnosis_{owner}_{repo}_{job_id}*, sanitised and length-capped, then gets or creates it with cosine distance. In a non-incremental run the collection is deleted first, so a re-run never mixes old vectors. For each file with chunks, it appends to a 128-chunk buffer; when the buffer fills, it requests embeddings, disambiguates duplicate identifiers, and adds documents with metadata. The final partial buffer follows the same path. Raw text is then released from state.

3.5.2.1 Dimension Guard.

On an incremental run, the stage peeks at one stored embedding. If its length differs from 2048, the collection is deleted and every file is re-embedded. This prevents a collection from mixing vectors produced by different models, which ChromaDB cannot query coherently. A changed embedding model therefore costs a full re-embed, not a partial one.

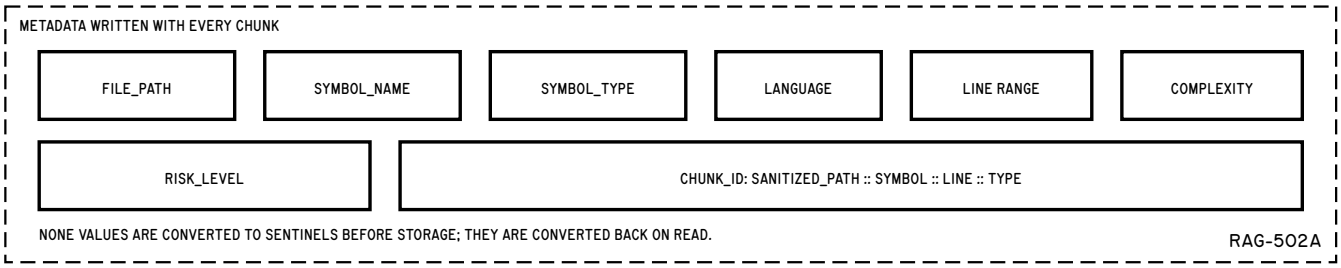


Figure 3.5-2. Chunk Metadata Contract

3.5.3 PERFORMANCE AND DESIGN DATA.

Table 3.5-1. Retrieval Parameters

Parameter	Value
Embedding model	nvidia/nemotron-3-embed-1b
Dimensions	2048
Embedding batch	50 texts
Stream buffer	128 chunks
Chroma add batch	128-chunk flush (500 constant unused)
Distance	cosine
Persistence	./chroma_db
Collection scope	owner + repo + job id

CAUTION

This stage clears *state.raw_contents* at the end. Stage 6's fallback therefore receives an empty string for every file, and explanation depends on retrieval succeeding. If the collection is empty or the retriever cannot connect, explanation degrades to a no-code prompt. See 8.1-1.

INDEX CHECK **MAINTAINER**

```
1 READ - collection name in state
2 CALL - retriever.count()
  √ COUNT - greater than zero
3 QUERY - a known function name
4 CONFIRM - file_path metadata matches
```

RAG

SECTION 3 · SUBSECTION 3.6
EXPLAINABILITY

3.6.1 INTRODUCTION.

Explainability is the only stage that calls a language model. It selects a bounded set of files, assembles code context from the vector index, builds a per-file prompt with structural metadata, and requests a short architectural explanation over an OpenAI-compatible endpoint. The one idea the reader must leave with is that selection, not generation, determines quality: only twenty files are explained by default, chosen by risk and import centrality (figure 3.6-1).

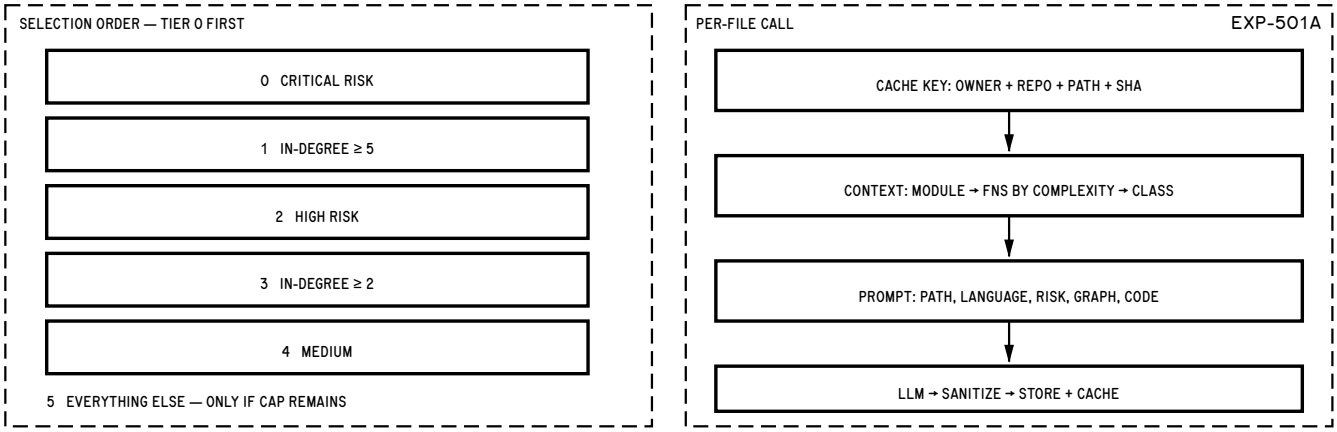


Figure 3.6-1. Selection Tiers and Per-File Call Sequence

3.6.2 FUNCTIONAL DESCRIPTION.

run requires a collection name and an *NVIDIA_API_KEY*; without either it returns the state unchanged. It connects a *CodeRetriever*, reuses explanations for unchanged files in incremental mode, and sorts candidates by tier, then descending in-degree, then path. For each selected file it checks a SHA-keyed disk cache, assembles up to 8,000 characters of chunk text, builds the prompt, and calls the model with temperature 0.1 and an 800-token limit. Successful answers are stripped, stored in state and cached. A failure is counted and skipped; it does not abort the run.

3.6.2.1 Context Budget.

Context assembly orders the module chunk first, then function chunks by descending complexity, then class chunks. It accumulates until 8,000 characters. If fewer than 300 characters remain, it writes a count of omitted chunks instead of a fragment. Retrieved code is therefore always cut at a chunk boundary or marked as truncated, never silently.

```
# src/agents/explainability.py, _assemble_code_context (condensed)
chunks = retriever.get_file_chunks(file_path)          # one file, no semantic query
if not chunks: return raw_content[:8000] if raw_content else ""   # normally empty
ordered = module_chunks + sorted(fn_chunks, key=lambda c: -(c['complexity'] or 0)) + class_chunks
for chunk in ordered:
    if char_count + len(chunk['content']) > 8000:      # hard budget
        parts.append(chunk['content'][:remaining] + "\n... [truncated]")
        break
return "\n\n---\n\n".join(parts)                       # visible separator
```

3.6.3 PERFORMANCE AND DESIGN DATA.

Table 3.6-1. Explanation Parameters

Parameter	Value
Default file cap	20
Allowed cap	0 to 100
Context budget	8,000 characters
High in-degree tier	≥ 5
Medium in-degree tier	≥ 2
Temperature	0.1
Max output tokens	800
Per-call timeout	120 s
Retries / backoff	2 attempts, 1 s base

CAUTION

The stage prints the first eight characters of the API key to its log at start-up. Treat logs as sensitive and remove that line before sharing output. The prompt also contains retrieved repository code; do not send private code to a hosted endpoint without authorisation.

EXPLAIN CHECK	OPERATOR
1 CONFIRM - NVIDIA_API_KEY set	
2 READ - files selected count	
√ CACHE - hit rate reported	
3 OPEN - one explanation	
4 CONFIRM - names real symbols	

EXP

SECTION 3 · SUBSECTION 3.7
DOCUMENT GENERATION

3.7.1 INTRODUCTION.

The document generator is a pure function over state. It assembles the onboarding document section by section, produces a compact context document for a downstream AI assistant, serialises the complexity report, and writes a per-file explanation pair in Markdown and JSON. The one idea the reader must leave with is that every section is independently guarded: one failed section is omitted with a warning, and the remaining document is still returned (figure 3.7-1).

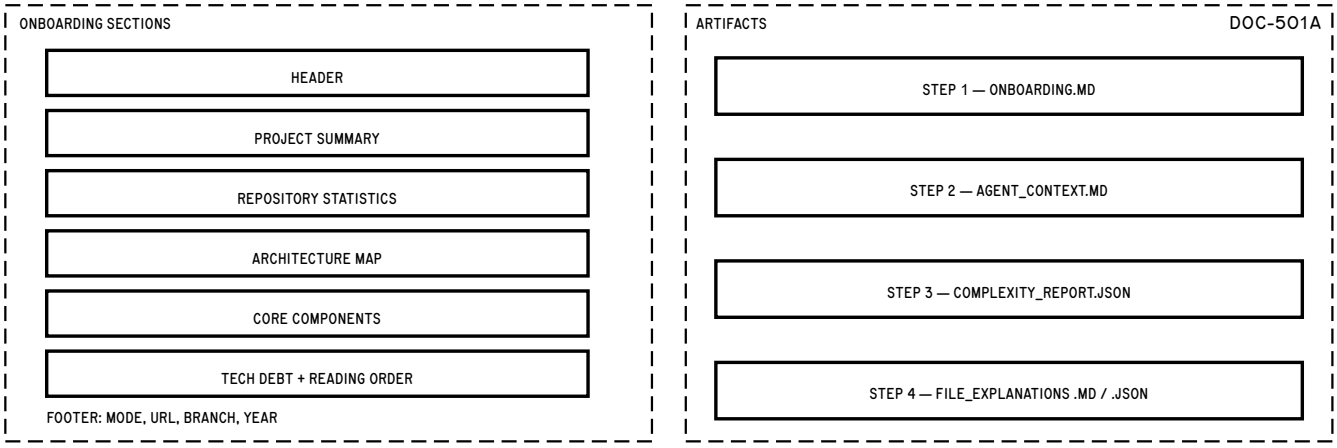


Figure 3.7-1. Document Assembly and Output Artifacts

3.7.2 FUNCTIONAL DESCRIPTION.

run iterates the onboarding builders in a fixed order, appending each returned string. It then calls *generate_agent_context_doc*, which composes ten sections aimed at an AI coding assistant: overview, entry points, core components, relationships, high-risk areas, cycles, domain objects, change impact, safe starting points and AI explanations. The complexity report is serialised from the score objects. The per-file explanation document and JSON are produced by separate builders. Each of the four major outputs is wrapped in its own try block.

3.7.2.1 Degradation Rule.

Section builders that fail are skipped, not replaced by a placeholder, so a document can be missing a heading without any visible marker in the body. A failure of the whole agent-context builder substitutes a one-line error document. The API always returns whatever fields were produced; the CLI writes only the files that exist.

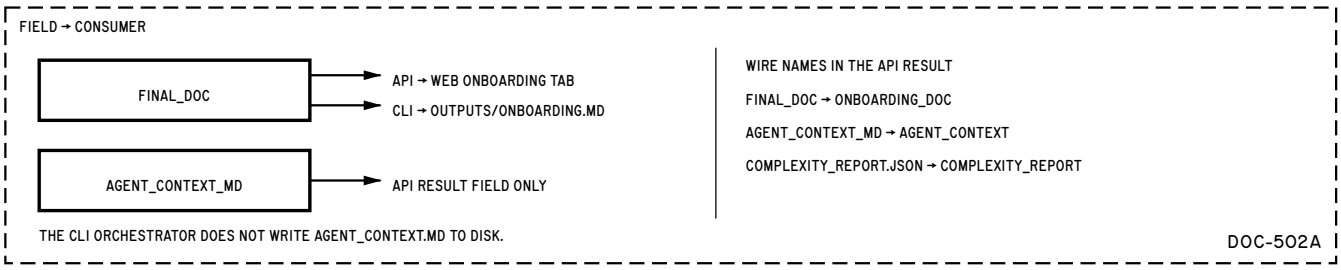


Figure 3.7-2. Output Field Consumers

3.7.3 PERFORMANCE AND DESIGN DATA.

Table 3.7-1. Generated Documents

Artifact	Content	Written by
onboarding.md	Eight narrative sections	CLI and API result
agent_context_md	Ten assistant-oriented sections	API result field only
complexity_report.json	Per-file scores and reasons	CLI and API
file_explanations.md	One section per explained file	CLI save helper
file_explanations.json	Path to explanation map	CLI save helper
symbol_tables.json	All parsed symbols	Orchestrator save
dependency_graph.json	Nodes, edges, cycles, order	Orchestrator save

CAUTION

The "Important Domain Objects" section reads *cls.methods*, but the class record exposes *method_names*. Every class therefore reports zero methods. See 8.1-1. Separately, the CLI path never persists *agent_context_md*; only the API returns it.

```
DOCUMENT CHECK                    REVIEWER
1 OPEN - onboarding.md
2 COUNT - major headings present
  √ RISK - table matches scores
3 CHECK - reading order not empty
4 VERIFY - footer names branch and mode
```

SECTION 3 · SUBSECTION 3.8
WEB API AND JOB CONTROL

3.8.1 INTRODUCTION.

The web API is a thin scheduling and status surface. It validates a submission, refuses duplicate active work, measures the repository before accepting it, creates a job record, and hands the job to either Redis Queue or a local thread pool. Every response is derived from the job store; the API holds no analysis result in memory of its own. The one idea the reader must leave with is that acceptance and execution are separate, so a 202 means scheduled, not finished (figure 3.8-1).

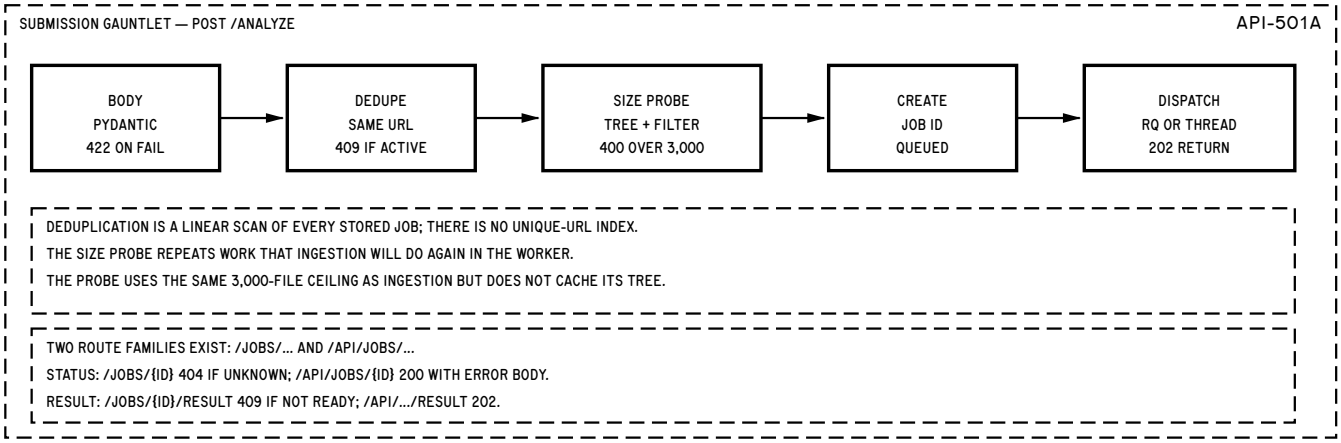


Figure 3.8-1. Submission Validation and Dual Route Behaviour

3.8.2 FUNCTIONAL DESCRIPTION.

The request model accepts *repo_url*, *max_explanations* (0 to 100), *skip_llm*, *github_token*, *incremental* and an optional *options* dictionary that can override the four execution fields. The URL validator requires the substring *github.com* and at least three path segments; it does not reject a non-GitHub host that contains the string. After deduplication, the route fetches repository metadata and the full tree to count filtered files, rejecting more than 3,000. It then creates the record and dispatches it. Status routes read the record; result routes return it or an error.

3.8.2.1 Synchronous Size Probe.

The probe runs inside the request, not in the worker. A large repository therefore delays the 202 response by the duration of two GitHub calls plus filtering, and a GitHub outage becomes a 400 with the text "Failed to validate repository size". The probe is a guard against wasting a worker slot, not a security boundary.

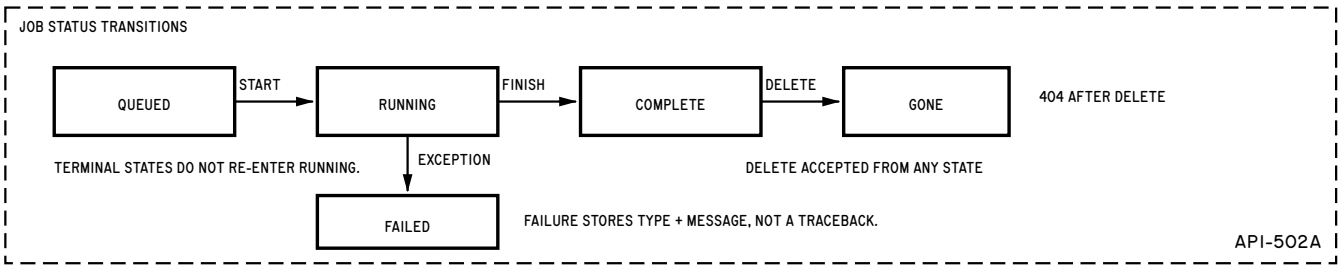


Figure 3.8-2. Stored Job Status Transitions

3.8.3 PERFORMANCE AND DESIGN DATA.

Table 3.8-1. HTTP Surface

Route	Success	Failure
POST /analyze	202 + job id	400, 409, 422
GET /jobs/{id}	200 record	404
GET /api/jobs/{id}	200 record	200 error body
GET /jobs/{id}/result	200 result	404, 409, 500
GET /api/jobs/{id}/result	200 result	202, 404, 500
GET /jobs	200 list	—
DELETE /jobs/{id}	204	404
GET /health	200 status	—

CAUTION

CORS allows every origin with credentials enabled, while the frontend also defines a strict content policy. The two settings serve different layers and do not cancel each other. Any site can call this API from a browser; do not place it behind assumptions of origin isolation.

API CHECK	OPERATOR
1 GET - /health, read queue mode	
2 POST - a small public repository	
√ 202 - job id returned	
3 POLL - /api/jobs/{id} advances	
4 FETCH - result after complete	

API

SECTION 3 · SUBSECTION 3.8 (CONT.)
STORE, QUEUE AND WORKER

3.8.4 INTRODUCTION.

Job state lives in one of two backends behind a single *JobStore* interface. When *REDIS_URL* is set and reachable, records are JSON strings in Redis, the job list is a sorted set, and an indexed set tracks active jobs. Otherwise the same interface uses a module-level dictionary and a lock. The one idea the reader must leave with is that Redis mode is not merely persistence: it adds a concurrency protocol that memory mode does not need (figure 3.8-3).

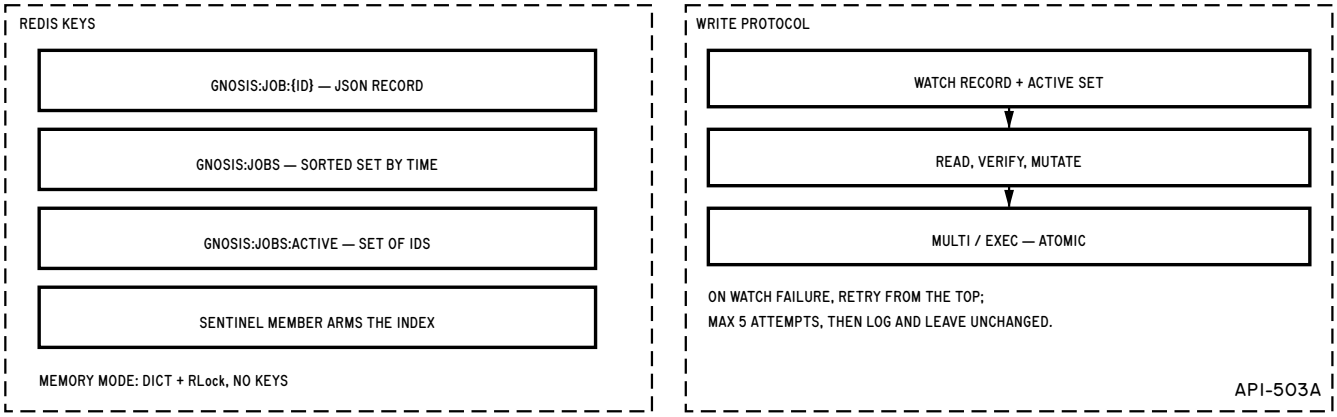


Figure 3.8-3. Redis Key Layout and Optimistic Write Loop

3.8.5 FUNCTIONAL DESCRIPTION.

create builds a queued record, writes it, adds its identifier to the list and, in Redis mode, to the active set. *start* and every phase update re-read the record inside a WATCH transaction, refuse to modify a terminal record, update fields, and remove the identifier from the active set on finish, fail or delete. If a competing writer changes the watched keys, the loop retries up to five times and then logs an error without partial writes. *active_count* reads the set cardinality, excluding the sentinel, and falls back to scanning records when the index is unarmed.

3.8.5.1 Index Reconciliation.

The active index can drift if a record expires, a process dies between two writes, or an operator clears a key. *rebuild_active_index* scans job records, rewrites the set, and arms it only after a complete pass. The migration command *job_store_migrate.py* refuses to arm unless the operator passes *--arm --confirm-all-writers-updated*; this prevents an old writer that does not maintain the set from corrupting the count.

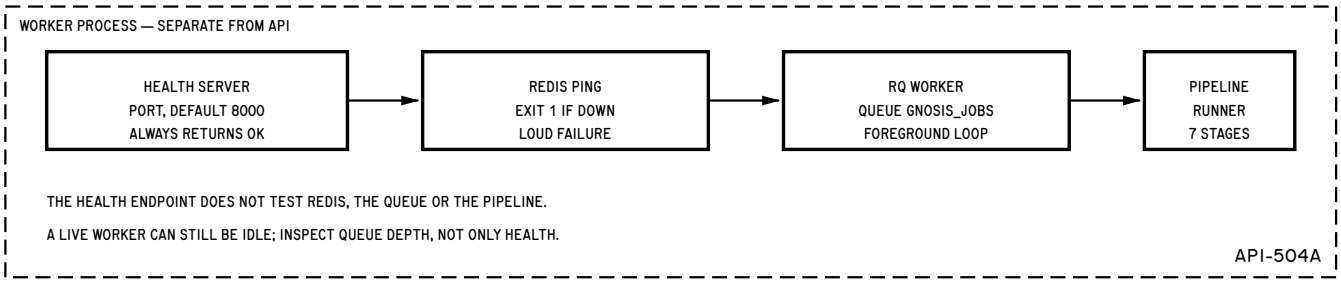


Figure 3.8-4. Worker Process and Its Health Surface

3.8.6 PERFORMANCE AND DESIGN DATA.

Table 3.8-2. Store, Queue and Worker Settings

Setting	Default	Effect
REDIS_URL	unset	Selects memory mode when absent
RQ_QUEUE_NAME	gnosis_jobs	Queue consumed by worker
JOB_TIMEOUT_SECONDS	1800	RQ execution limit
JOB_TTL_SECONDS	86400 / 0	RQ result TTL; store record TTL is 0
WATCH_MAX_ATTEMPTS	5	Optimistic write retries
ACTIVE_INDEX_SENTINEL	fixed string	Marks an armed index
Worker PORT	8000	Health listener
Local executor	2 threads	Fallback concurrency

CAUTION

Queue results expire after 86,400 seconds, but completed job records have no expiry unless *JOB_TTL_SECONDS* is also set in the API process. With the documented default the two layers diverge: the record survives while the RQ result is gone. Choose one retention policy.

STORE CHECK	OPERATOR
1 READ - /health queue_mode	
2 CONFIRM - redis_connected true	
√ DEPTH - worker draining	
3 CREATE - job reaches running	
4 VERIFY - active count returns to zero	

API

SECTION 3 · SUBSECTION 3.9
OPERATOR DISPLAY

3.9.1 INTRODUCTION.

The operator display is a single-page React application in *frontend/*. It has one job: submit a repository address, show which of the seven stages is running, then present the returned documents in six tabs. Navigation is a local state variable, not a URL router. The bundle is built by Vite and served as static files; every dynamic request goes to the FastAPI origin named by *VITE_API_URL* (figure 3.9-1).

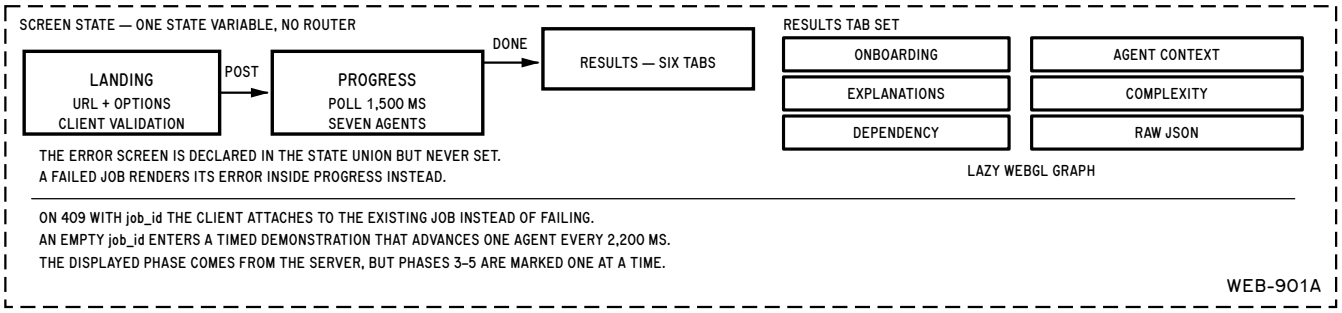


Figure 3.9-1. Client Screen Transitions and Result Tabs

3.9.2 FUNCTIONAL DESCRIPTION.

The landing screen accepts a repository address and two options: an explanation cap and a skip-LLM switch. It rejects anything that does not begin with *http(s)://github.com/owner/repo* before calling the API. A successful submission returns a job identifier; the client switches to the progress screen and requests */api/jobs/{id}* immediately, then every 1,500 ms. A non-OK response is ignored and polling continues, which makes the display tolerant of a restarting server. Terminal status stops the timer; unmounting clears it.

Progress is computed on the client: completed agents plus half credit for the running agent, divided by seven. The agent names and statuses themselves come from the API. When the job completes, the results screen fetches */api/jobs/{id}/result* once and renders the Markdown, the JSON and the graph payload. The 3D graph module is split into a lazy chunk so the first paint does not pay for *three* and *3d-force-graph*.

3.9.2.1 Document Presentation.

Markdown is rendered with a GitHub-flavoured parser and a table extension. Downloads are produced in the browser from the strings already in memory, so the API has no download endpoint. Raw JSON exposes the serialised result for inspection. The dependency tab builds a force-directed graph from nodes and links; node colour is the risk level, and the circular-dependency flag is shown in the side panel.

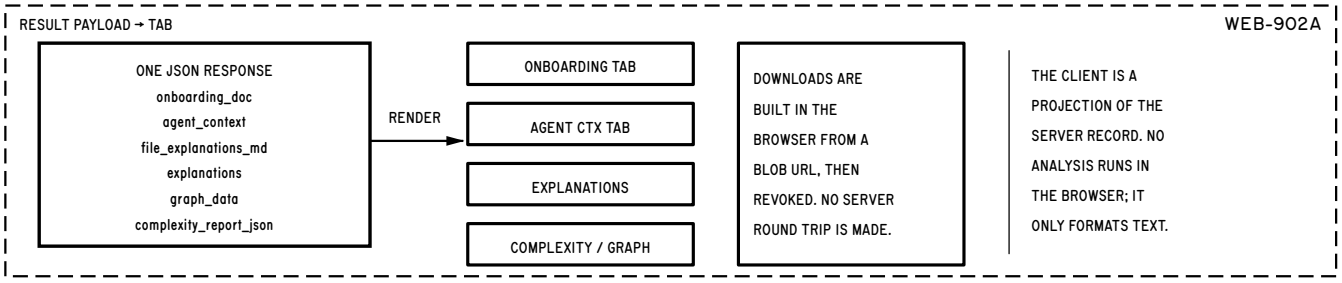


Figure 3.9-2. Single Payload, Client-Side Rendering and Download

3.9.3 PERFORMANCE AND DESIGN DATA.

Table 3.9-1. Client Runtime Constants

Item	Value	Source
API origin	VITE_API_URL or same origin	App.tsx
Poll interval	1,500 ms	App.tsx
Demo tick	2,200 ms	App.tsx
Agents displayed	7, server supplied	/api/jobs/{id}
Result tabs	6	App.tsx
App version string	1.1.0	App.tsx
3D graph bundle	lazy chunk	React.lazy
Router dependency	installed, unused	package.json

CAUTION

The client trusts the server's agent list to describe concurrency. The API marks graph, complexity and retrieval as separate sequential phases even while their futures overlap, so the display understates parallelism. The progress percentage is therefore a presentation estimate, not a measured completion fraction.

DISPLAY CHECK	OPERATOR
1 SUBMIT - bad URL rejected	
2 SUBMIT - duplicate job attaches via 409	
√ POLL - seven agent rows update	
3 OPEN - all six result tabs	
4 VERIFY - downloads open offline	

SECTION 4 · SUBSECTION 4.1
END-TO-END LOGIC FLOW

4.1.1 INTRODUCTION.

A run is one traversal of the shared state object. The orchestrator resolves the repository address, asks GitHub for the default branch, then hands the same *ArchaeonState* to each agent in turn. Agents never import one another; they only read fields and write new ones. The only departure from sequence is the middle group: graph, complexity and retrieval are submitted to a three-thread pool, then joined in a fixed order so that graph statistics can be bound into the complexity scores before they are consumed (figure 4.1-1).

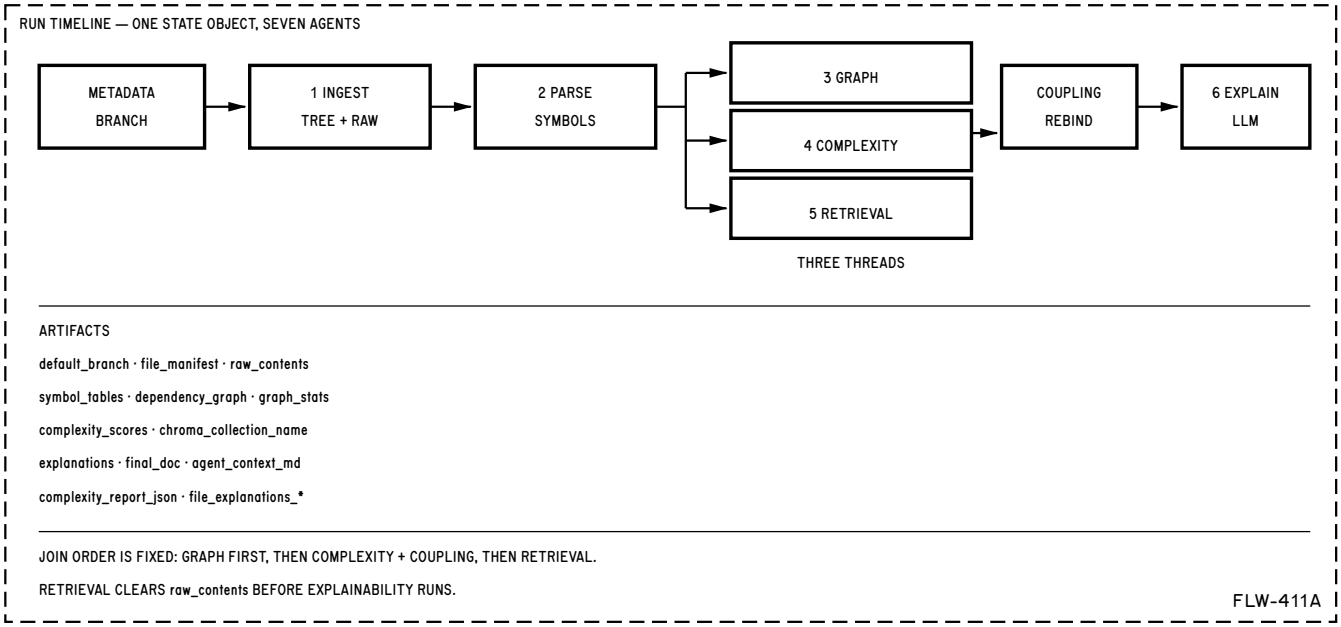


Figure 4.1-1. Phase Order, Fan-Out and State Artifacts

4.1.2 FUNCTIONAL DESCRIPTION.

Metadata resolution is a single *GET /repos/{owner}/{repo}*. Ingestion then lists the default branch recursively, filters paths by extension, excluded directory and size, and fetches raw text from *raw.githubusercontent.com*. The manifest is sorted by depth then path before any cap is applied. Parsing builds one symbol table per file, recording functions, classes, imports, docstrings and parse errors; a parse failure is data, not an abort.

The middle group starts with all three futures alive. The orchestrator waits for the graph, then for complexity, immediately calls *apply_graph_coupling* to replace placeholder coupling values with real in-degree and out-degree, and only then waits for retrieval. Retrieval chunks each parsed symbol, embeds the chunks in request batches and writes them to a per-job Chroma collection. After the join, explainability ranks files and asks the hosted model for prose; the document generator assembles four outputs and returns control.

4.1.2.1 CLI and API Divergence.

The CLI calls *run_pipeline* and then writes ten files itself, one save function at a time. The API calls the same function inside *pipeline_runner*, serialises a subset of state into one JSON record, and never writes files. The API result therefore omits nothing the browser needs but does omit *symbol_tables* details, and the CLI writes *file_explanations* artifacts that the API returns only inside the aggregate result.

4.1.3 PERFORMANCE AND DESIGN DATA.

Table 4.1-1. Phase Inputs, Outputs and Limits

Phase	Reads	Writes
Metadata	owner, repo, token	default_branch
1 Ingestion	URL, branch	manifest, raw text
2 Parse	raw text, paths	symbol tables
3 Graph	imports, manifest	graph, cycles, order
4 Complexity	raw text, symbols	scores, risk
5 Retrieval	raw text, symbols	collection, chunks
Coupling	graph stats	coupling, final risk
6 Explain	scores, graph, Chroma	explanations
7 Document	all of the above	four documents

NOTE

Retrieval empties *raw_contents* on success to release memory. Explainability's fallback path then finds no raw text and must rely on Chroma chunks, which exist. If retrieval stored zero chunks it returns before clearing, so the fallback still has text but the collection is empty.

FLOW CHECK	REVIEWER
1 TRACE - manifest count to parse count	
2 CONFIRM - graph waited before coupling	
√ JOIN - retrieval after coupling	
3 CHECK - explanations reference real files	
4 VERIFY - document headings not empty	

FLW

SECTION 4 · SUBSECTION 4.2
STATE, ERRORS AND RETRIES

4.2.1 INTRODUCTION.

Failure handling is not uniform. Some faults abort the whole job, some degrade one file, and some are retried inside a client. The dividing line is whether the remaining stages can still produce a truthful document. A missing repository tree cannot be worked around; a single unparsable file can be reported as such; a failed model call can leave one file unexplained. The operator must know which class a given error belongs to before interpreting an incomplete result (figure 4.2-1).

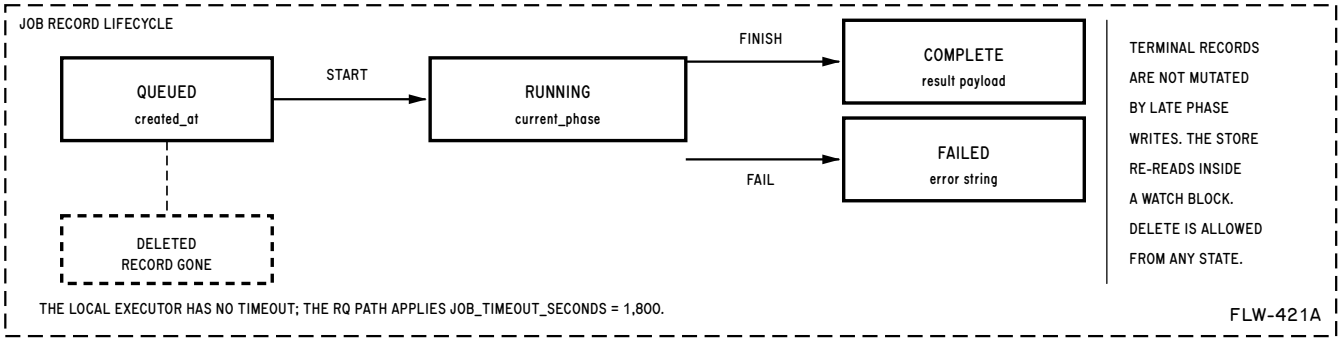


Figure 4.2-1. Queued, Running and Terminal Job States

4.2.2 FUNCTIONAL DESCRIPTION.

In API mode, *pipeline_runner* wraps the entire sequence in one exception handler. Any uncaught exception, including one raised while waiting on a future, is formatted as *Type: message*, the traceback is printed to the server log, and the record moves to FAILED with that string. The HTTP result endpoint then answers 500 for that job. The CLI has no equivalent wrapper: an uncaught exception ends the process, and whatever files were written before the failure remain.

Within the sequence, containment varies. Parse errors are stored in the symbol table with a detail string and later force a CRITICAL risk level. A failed PageRank converges to normalised in-degree, and a total failure yields zero scores rather than an exception. Explainability returns early when the collection or API key is absent and skips individual files whose call returns nothing. The document generator wraps each section, and each output, in its own handler so one bad section does not remove the rest.

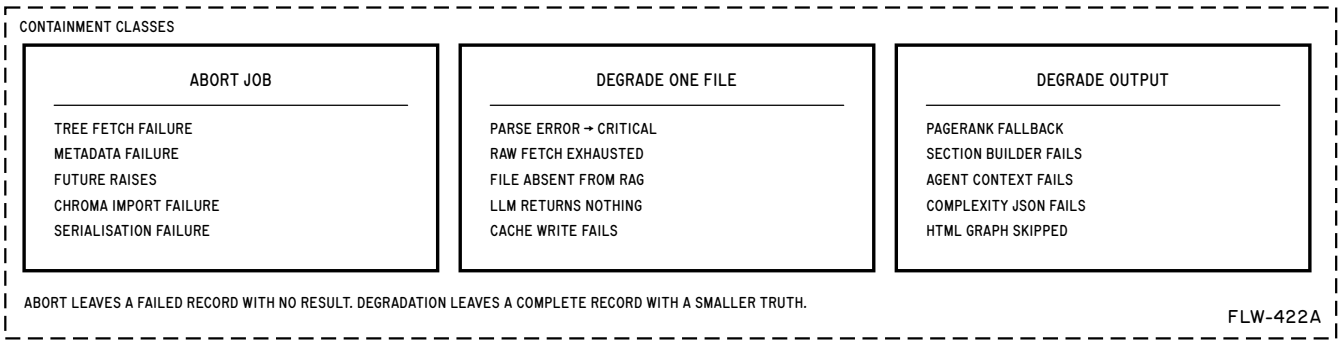


Figure 4.2-2. Which Faults Stop a Run and Which Only Weaken It

4.2.3 PERFORMANCE AND DESIGN DATA.

Table 4.2-1. Retry and Timeout Boundaries

Operation	Attempts	Backoff / limit
Raw file fetch	5	fixed 0.05 s inter-request
GitHub metadata/tree	1	raises to caller
Embedding request	50 texts per call	0.5 s between batches
Chat completion	2	120 s per attempt; 2 s pause
Store WATCH write	5	abort with error
RQ execution	1	1,800 s job timeout
Local thread pool	1	no timeout
Frontend poll	indefinite	1,500 ms; ignores errors

WARNING

Two latent faults escape their intended containment. Incremental chunk deletion calls an undefined *logger*, so a failed delete raises *NameError* instead of being logged. The CLI never writes *agent_context_md* to disk, so a successful run still lacks that deliverable. Neither is exercised by the current suite.

FAILURE CHECK	INVESTIGATOR
1 READ - status and error string	
2 CLASSIFY - abort, file or output	
√ COUNT - explained vs selected	
3 CHECK - parse-error list	
4 CONFIRM - no silent sample	

SECTION 5 · SUBSECTION 5.1
FILE CATALOGUE: BACKEND

5.1.1 INTRODUCTION.

This catalogue covers every tracked file at the analysed commit, plus two local files that are present but not yet tracked. It excludes generated and vendored material: the virtual environment, *node_modules*, build output, Chroma persistence, run outputs, caches and this handbook. Line counts are non-empty lines at the working tree, not logical statements. The purpose is orientation: a reader should be able to name the file that produces any value seen in the output (figure 5.1-1).

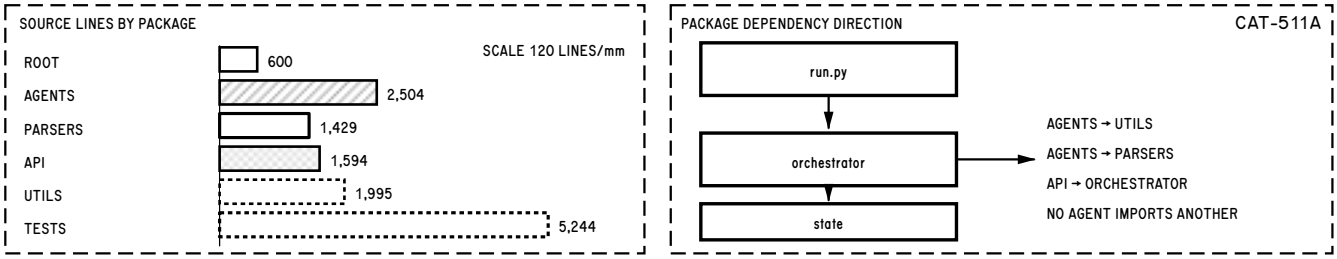


Figure 5.1-1. Package Size and Allowed Import Direction

5.1.2 BACKEND MODULES.

Thirty-three Python modules carry the backend. *src/parsers* turns syntax into dataclasses; *src/utils* supplies everything that talks to the outside world; *src/agents* holds the seven stages; *src/api* adds transport and persistence. Empty *__init__.py* files are omitted from the table.

Table 5.1-1. Backend Module Inventory (empty lines excluded)

Module	Lines	Responsibility
run.py	204	Argument parsing, size pre-check, pipeline call, ten save calls
src/orchestrator.py	335	Phase order, three-thread join, coupling rebind, output writers
src/state.py	61	*FileMetadata* and *ArchaeonState*; the only cross-agent contract
agents/ingestion.py	135	Filter, sort, tier, cap at 3,000, batch fetch, line counts
agents/ast_parser.py	335	Parallel parse, language dispatch, import internals, parse errors
agents/dependency_graph.py	254	Nodes, edges, cycles, PageRank, topological order, per-file stats
agents/complexity_scorer.py	353	Function scores, thresholds, risk reasons, deferred coupling
agents/code_rag.py	254	Chunking, batched embeddings, Chroma write, raw-text release
agents/explainability.py	373	File ranking, context budget, prompt, chat call, explanation cache
agents/doc_generator.py	800	Onboarding, agent context, explanations, complexity JSON; section guards
api/main.py	322	Dual route families, CORS, headers, request validation, queue choice
api/models.py	142	Pydantic request, status, summary and result contracts
api/pipeline_runner.py	295	API phase sequence, progress marks, result serialisation, failure record
api/job_store.py	552	Memory and Redis stores, WATCH loop, active-index reconciliation
api/job_store_migrate.py	95	Operator command to rebuild and arm the active index (untracked)
api/queue.py	107	Redis connection, RQ queue, enqueue, depth and worker statistics
api/worker.py	81	Health listener, Redis probe, RQ foreground worker
parsers/base.py	91	*FunctionInfo*, *ClassInfo*, *ImportInfo*, *SymbolTable*, *ComplexityScore*
parsers/python_parser.py	354	Python functions, classes, decorators, imports, docstrings
parsers/js_parser.py	285	JavaScript and TypeScript declarations, arrows, imports
parsers/generic_parser.py	426	Go, Rust, Java, C and C++ symbol extraction
parsers/complexity.py	273	Python and JavaScript branch counting; generic fallback
utils/github_api.py	197	URL parsing, metadata, tree, raw fetch pool, API fallback
utils/filters.py	109	Extension allow-list, 24 excluded directories, 150 KB ceiling
utils/tree_sitter_utils.py	88	Language grammar loading and parser cache
utils/chunker.py	223	Module, class and function chunks with stable identifiers
utils/embedder.py	114	Embedding client, batching, passage and query input types
utils/retriever.py	233	Collection naming, cosine query, metadata filters, parsing
utils/nvidia_client.py	208	Chat client, timeout, two attempts, output sanitisation
utils/explanation_cache.py	135	Content-addressed explanation cache on local disk
utils/diff_engine.py	242	SHA-based manifest diff and state rehydration
utils/graph_utils.py	387	Per-language import resolution and pyvis HTML export
utils/metrics.py	59	Phase timing and resident-memory prints

NOTE

Two files are outside the commit. *job_store_migrate.py* and *test_active_index.py* exist only in the working tree and are marked accordingly. Their presence means the committed source cannot rebuild the active index, even though the store can fall back to scanning records.

CATALOGUE CHECK	REVIEWER
√ COUNT - 33 modules named	
√ LINES - non-empty, working tree	
1 EXCLUDED - env, deps, output, caches	
2 MARKED - untracked files	

CAT

SECTION 5 · SUBSECTION 5.1 (CONT.)
FILE CATALOGUE: TESTS AND RECORDS

5.1.3 INTRODUCTION.

The repository's second half is evidence: 5,244 lines of tests and roughly 2,200 lines of tracked records and configuration. The test suite is the strongest description of intended behaviour in the tree, and it is more current than the README. It runs entirely offline: HTTP, Redis, Chroma and the hosted model are replaced by fakes, *TestClient*, *fakeredis* and monkeypatching. No test contacts GitHub, NVIDIA or a live Redis instance. The design and memory files marked *local, ignored* exist in this working tree but in no clone (figure 5.1-2).

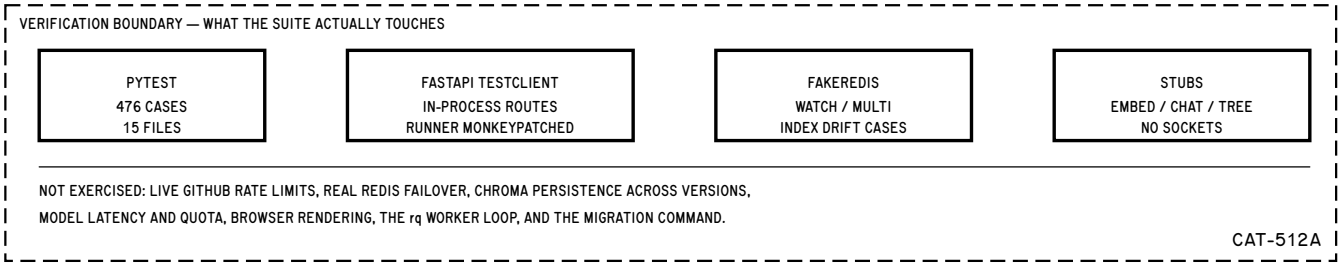


Figure 5.1-2. Offline Test Boundary and Uncovered Surfaces

5.1.4 TEST FILES.

Counts are collected cases, not assertions. The four heaviest files cover document rendering, HTTP contracts, retrieval and explanations.

Table 5.1-2. Test Inventory, 476 Collected Cases (empty lines excluded)

File	Cases	Lines	Focus
test_doc_generator.py	78	641	Section presence, ordering, empty-state robustness, heading hierarchy
test_api.py	75	738	Routes, status codes, store lifecycle, health, validation, 409 duplicate
test_code_rag.py	56	653	Chunk identity, collection names, filters, embedder batching, results
test_complexity_scorer.py	50	377	Branch counting per language, thresholds, precedence, coupling
test_explainability.py	47	523	Selection order, context budget, prompt fields, sanitisation, failures
test_dependency_graph.py	38	421	Python/JS/Rust/C/Go resolution, cycles, order, PageRank fallback
test_ast_parser.py	34	291	Python and JS symbols, docstrings, relative imports, parse errors
test_active_index.py	31	519	Membership, contention, drift, reconciliation, migration refusal (untracked)
test_ingestion.py	22	61	URL forms, extension and directory rules, size ceiling, language labels
test_multi_language.py	15	430	Go, Rust, Java and C++ symbols and complexity; alias resolution
test_queue_and_store.py	15	192	Job round-trip, memory store, fakeredis store, enqueue, worker health
test_tiers.py	7	164	Full, warning and sampled modes; API rejection above 3,000
test_github_api_pat.py	3	47	Bearer header, private-repository message, raw-to-API fallback
test_incremental.py	3	125	Manifest diff classes and rehydration of symbols and scores
test_parallel_execution.py	2	62	Concurrent parse and score calls produce the same state

Table 5.1-3. Records and Repository Configuration (empty lines excluded)

File	Lines	Status	Content
README.md	498	tracked	User-facing overview; several claims now stale
PROJECT_SPEC.md	424	tracked	Original specification and phase plan
GLOSSARY.md	169	tracked	Domain and pipeline vocabulary
agent.md	166	tracked	Code-verified handoff; most reliable prose record
LICENSE	544	tracked	AGPL-3.0 with commercial option
DESIGN.md	1,244	local, ignored	Design system: colour, type, components, motion
Decisions.md	437	local, ignored	Numbered decision log; duplicate identifiers exist
ChangeLog.md	348	local, ignored	Release history; version order is not monotonic
Flow.md	105	local, ignored	Agent-oriented flow summary
ROADMAP.md	41	local, ignored	Candidate work, not commitments
SCRATCHPAD.md	46	local, ignored	Working notes and obstacles
GEMINI.md	16	local, ignored	Workspace rules for AI assistants
.env.example	37	tracked	Variable names only; no values
requirements.txt	26	tracked	Unpinned lower bounds; no hash lock
Dockerfile	17	tracked	Python 3.11-slim image
docker-compose.yml	45	tracked	API, worker and Redis services
.dockerignore	26	tracked	Build-context exclusions
.gitignore	38	tracked	Runtime, memory and secret exclusions
vercel.json	52	tracked	Root redirect and proxy rules
workflows/...-api.yml	76	tracked	Azure App Service deployment for the API
workflows/...-worker.yml	66	tracked	Azure deployment for the worker

CAT

SECTION 5 · SUBSECTION 5.2
FILE CATALOGUE: FRONTEND AND ASSETS

5.2.1 INTRODUCTION.

The frontend is a Vite project nested inside the backend repository. Eight files under *src/* matter; one of them, *pages/ResultsPage.tsx*, is a complete earlier implementation that nothing imports. The rest is build configuration, brand imagery and an *imports* folder preserving the Figma hand-off; those archives are not compiled into the application but are retained as design provenance (figure 5.2-1).

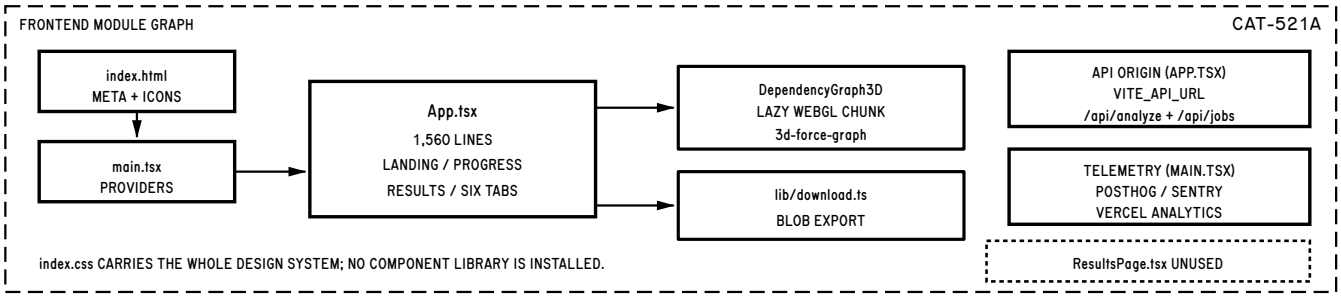


Figure 5.2-1. Compiled Frontend Modules and External Endpoints

5.2.2 APPLICATION SOURCE AND CONFIGURATION.

Table 5.2-1. Frontend Source, Build and Deployment Files (empty lines excluded)

File	Lines	Role
src/App.tsx	1,560	All four screens, polling, six result tabs, inline design components
src/components/DependencyGraph3D.tsx	203	WebGL force graph; nodes coloured by risk; resize handling
src/pages/ResultsPage.tsx	762	Earlier results implementation; imported by nothing
src/index.css	378	Design tokens, typography, layout, Markdown and scrollbar styling
src/types/api.ts	87	Hand-written mirror of the Pydantic result contract
src/lib/download.ts	37	Create, click and revoke a Blob download link
src/main.tsx	32	Root render, Sentry initialisation, PostHog provider, analytics
src/vite-env.d.ts	1	Vite client type reference
index.html	43	Canonical URL, Open Graph and Twitter metadata, icons
vite.config.ts	335	React and Tailwind plugins, aliases, dev proxy, Figma host shim
package.json	38	Dependencies and four scripts; package name still *figma-make-app*
package-lock.json	lock	Resolved dependency tree; the only frontend lockfile
tsconfig.json	23	Bundler resolution, strictness, JSX settings
vercel.json	52	PostHog reverse proxy and security headers
.env.development / .env.production	3 each	Analytics project key and ingest host; values not reproduced here
.figma/make/site.json	3	Figma Make site descriptor

Table 5.2-2. Public Assets (Size in KB)

Asset	KB	Asset	KB
favicon-16x16.png	1	logo.png	598
favicon-32x32.png	1	logo_transparent.png	696
favicon.ico	2	logo_black.png	69
apple-touch-icon.png	14	logo_blue.png	69
android-chrome-192x192.png	15	logo_white.png	69
android-chrome-512x512.png	53	og-image.png	148
og-image.jpg	84	pitch_deck_logo.png	69
"pitch deck logo.png"	69	pitch_deck_logo_navy.png	69
pitch_deck_logo_white.png	69	pitch_deck_logo_navy_card.png	99
pitch_deck_logo_paper.png	98	—	—

Table 5.2-3. Import Archives and Vendored Browser Assets (empty lines excluded)

Path	Lines	Size	Content
src/imports/README.md	405	text	Earlier project readme preserved with the design hand-off
src/imports/PROJECT_SPEC.md	369	text	Earlier specification; differs from the root copy
src/imports/backend/api_main.py	198	text	Earlier API sketch; not imported by any runtime path
src/imports/backend/orchestrator_patch.py	65	text	Patch fragment from the initial build
src/imports/pasted_text/*.md	754 ea.	text	Two identical UI design transcripts
src/imports/image.png	—	1,913 KB	Large reference screenshot
src/imports/image-1.png	—	26 KB	Second reference screenshot
lib/vis-9.1.2/vis-network.min.js	27	458 KB	Graph engine used by generated pyvis HTML
lib/vis-9.1.2/vis-network.css	1	215 KB	Matching stylesheet
lib/tom-select/ (.js + .css)	655	53 KB	Select control required by pyvis templates
lib/bindings/utils.js	175	6 KB	pyvis browser helpers

CAT

SECTION 6 · SUBSECTION 6.1
DESIGN DECISIONS

6.1.1 INTRODUCTION.

This section separates what the code does from what the records say about why. Each entry carries a basis mark. **OBSERVED** means the behaviour is directly visible in the working tree. **DOCUMENTED** means a repository record states the rationale, usually the decision log. **INFERRED** means the consequence or motive is this handbook's reading and is not asserted by the project. The distinction matters because several remembered decisions no longer match the implementation (figure 6.1-1).

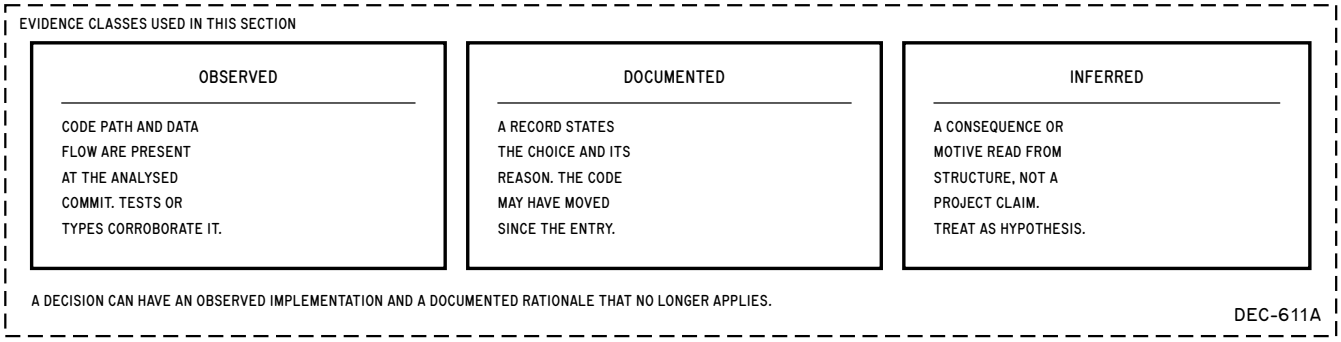


Figure 6.1-1. How Each Claim in This Section Is Grounded

DEC

6.1.2 ARCHITECTURAL DECISIONS.

The system is organised around one mutable state object and a fixed stage order. That choice keeps every intermediate value inspectable and makes the CLI and API share a single implementation, but it also couples all stages to one schema and makes partial re-runs coarse.

Table 6.1-1. Principal Decisions and Their Consequences

Decision	Basis	Consequence and cost
One shared state object	OBSERVED	Every stage is independently inspectable; schema changes ripple through all agents; no message contract or versioning
Agents do not import agents	OBSERVED	Stage replacement is local; orchestration knowledge concentrates in two runners that can drift
Parallel graph, complexity, retrieval	OBSERVED / DOCUMENTED	Wall-clock saving on I/O-bound retrieval; coupling must be deferred and reapplied after the join
Depth-first file cap	OBSERVED	Shallow entry points survive a sample; deep but central modules can be excluded without a warning in the document body
Hosted embeddings and chat	OBSERVED / DOCUMENTED	No local GPU or model download; the tool stops working without network, key and quota
Chroma collection per job	OBSERVED	Jobs cannot collide or read each other's code; collections accumulate because no retention job removes them
Redis optional, memory fallback	OBSERVED	One-command local start; memory mode cannot share state across processes and loses jobs on restart
Dual route families	OBSERVED / INFERRED	Older clients keep working; the two families return different codes and error shapes for the same condition
Markdown as primary output	OBSERVED / DOCUMENTED	Readable in any editor and usable as model context; structure is prose, so machine consumers must parse it or use JSON
Fail-open document sections	OBSERVED / INFERRED	A partial document is still delivered; a missing section can look like an empty finding rather than a failure
Client-side rendering and export	OBSERVED	No server download endpoint and no result storage beyond the job record; large payloads cross the network once
No URL router in the SPA	OBSERVED	Refresh returns to the landing screen; deep links and browser history are unavailable despite the installed router

6.1.3 Why the Pipeline Is Not a DAG Scheduler.

The three middle stages are independent after parsing, so a general scheduler was unnecessary. A fixed pool with explicit joins gives deterministic ordering of writes and avoids concurrent mutation of unrelated state fields. The cost is that a slow retrieval stage delays explainability even when the graph and scores are ready, because the join is a barrier. A finer-grained dependency graph could start explanation for files whose scores are final, but it would require per-file readiness tracking that the state object does not carry.

CAUTION

Per-job Chroma collections are removed only when *DELETE /jobs/{id}* is called, and that call is best-effort: a failure is logged and the record is deleted anyway. Nothing prunes on a timer and the client never calls it, so every analysis leaves an on-disk collection. Disk growth tracks runs, not repositories.

DECISION CHECK	REVIEWER
1 ASK - what does the code do	
2 ASK - what does the record claim	
√ MARK - observed, documented, inferred	
3 NAME - the cost, not only the benefit	

SECTION 6 · SUBSECTION 6.2
DECISION LEDGER AND DOCUMENT DRIFT

6.2.1 INTRODUCTION.

The repository carries three layers of description: the public README, the original specification, and the decision and change logs. They were written at different times and are not regenerated. Where they disagree with the code, this handbook follows the code and records the disagreement. The README is the most visible and the most stale; *agent.md* is the most current prose record; the decision log is the richest rationale but contains numbering faults (figure 6.2-1).

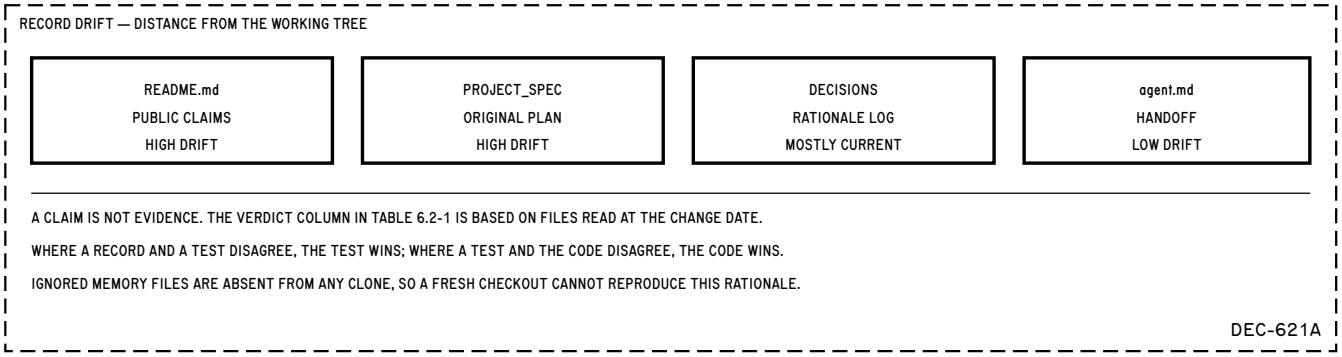


Figure 6.2-1. Description Layers Ordered by Distance from Code

DEC

6.2.2 CLAIM-BY-CLAIM LEDGER.

The left column quotes or paraphrases the record. The right column states what the tree actually contains. Only disagreements are listed; agreements are omitted rather than padded.

Table 6.2-1. Documented Claims Checked Against the Working Tree

Record claim	Source	Working tree
"Sequential multi-agent pipeline"; parallel is "a v2 optimization"	README 65, 567	Graph, complexity and retrieval run in a three-thread pool; coupling is deferred to a post-join fix-up
Ingestion uses "max 15 threads"	README 198, 608	Pool size is *min(16, total files)*; delay 0.05 s; five retries per file
"407 tests" in CI	README 220	476 cases collected and passing on the local interpreter
Frontend is "React 18 + … shadcn/ui"	README 205, 346	Resolved React 19.2.8, Vite 8.2.0, Tailwind 4.3.3; no shadcn package or components
All job state is a module dict; Redis is a future upgrade	README 584	Redis store, RQ queue and worker are implemented; memory mode is only the fallback
Workflow lives at *.github/workflows/azure-deploy.yml*	README 459	Two files: *master_project-gnosis-api.yml* and *master_project-gnosis-worker.yml*
Test fixture repository at *tests/fixtures/sample_repo/*	README 364	The directory is empty and untracked; no test reads it, and tests build strings and fakes in memory
Seven phases run "in sequence" in the entry-point docstring	run.py header	Entry point is sequential; the orchestrator is partially parallel
Public release is v1.0.0; version string in the client is 1.1.0	ChangeLog / App.tsx	Two different *1.1.0* releases and an out-of-order *1.0.8*; no single source of version truth
"Deploys automatically on every push to master"	README 455	Workflows exist, but trigger paths and branch filters are defined per workflow and were not executed here
Python 3.11 in the manual Azure command	README 464	Container image uses 3.11-slim; both Azure workflows request 3.12; local interpreter is 3.11.15
Decision log identifiers are unique and ordered	Decisions.md	*DEC-008* appears twice with different subjects; entries are non-monotonic

6.2.3 Records That Remain Useful.

agent.md is the best starting point for a code-aware reader: it was written against the current pipeline and lists its own known drift. *Decisions.md* remains valuable for rationale even where identifiers are duplicated, because each entry names the problem it solved. *DESIGN.md* describes a visual system that is largely present in *index.css* and *App.tsx*, but it is ignored by git and therefore not part of a clone. *GLOSSARY.md* and *PROJECT_SPEC.md* describe the intended vocabulary and original scope; they should be read as history.

WARNING

The README's deployment section names a workflow file that does not exist. A reader following it will look for the wrong path and may conclude deployment is unconfigured. The actual workflows are present under different names; their live status was not tested and is not asserted here.

DRIFT CHECK	MAINTAINER
1 PICK - one checkable claim	
2 OPEN - the file that would prove it	
√ RECORD - code, test, doc, or absent	
3 FIX - the record, not the code, by default	
4 DATE - every correction	

SECTION 7 · SUBSECTION 7.1
STARTING AND OPERATING

7.1.1 INTRODUCTION.

There are two deployment shapes and they differ in more than transport. The local shape runs the API in one process and executes jobs on a thread pool inside it. The distributed shape adds Redis and at least one worker process; the API only enqueues. Both share the same store interface and the same pipeline function. For reading or testing the codebase, the local shape needs no Redis and no model key if the LLM stage is skipped (figure 7.1-1).

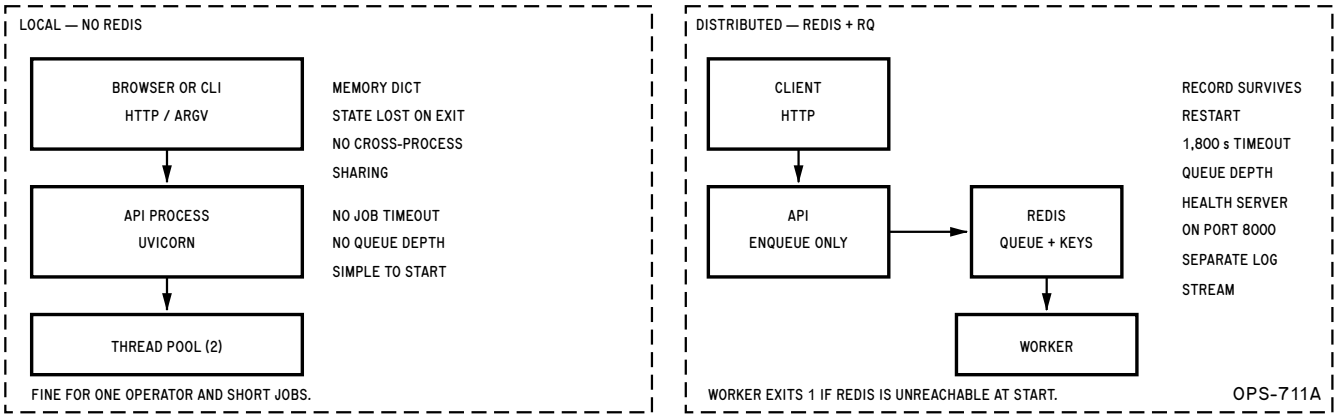


Figure 7.1-1. Process Topology With and Without Redis

7.1.2 FUNCTIONAL DESCRIPTION.

A CLI run needs an interpreter, dependencies, a GitHub address and, unless *--skip-llm* is passed, an NVIDIA key. It writes ten files into the output directory. An API run needs the same environment plus an HTTP client; when Redis is absent, the API executes the pipeline in a two-thread local pool. The frontend is independent: it needs Node, a build, and the API origin.

Environment is loaded from *.env* by *python-dotenv* in both entry points. Variable names below come from the example file only; no values were read for this handbook. A private repository requires a token, which is propagated per request and never persisted in a job record.

7.1.2.1 Commands Executed for This Handbook.

The commands below were run in the working tree on the change date. Nothing else was executed: in particular, no remote analysis, no Docker build, no worker start and no frontend build. Results are reported exactly as observed.

Table 7.1-1. Verified Commands and Observed Results

Command	Observed result
.venv\Scripts\python.exe -m pytest tests -q	476 passed in 20.40 s
.venv\Scripts\python.exe -m pytest --collect-only -q	476 cases across 15 files
.venv\Scripts\python.exe run.py --help	Usage printed; exits 0; options *--url*, *--token*, *--incremental*, *--output*, *--no-html*, *--skip-llm*, *--max-explain*, *--test-rag*
.venv\Scripts\python.exe -c "import src.api.main"	Imports without error; no startup side effects observed
python -m uvicorn src.api.main:app --port 8011	Startup complete; listener on 127.0.0.1:8011
GET http://127.0.0.1:8011/health	*status ok*, version 1.0.0, active_jobs 0, redis_connected false, queue_mode local_threadpool, depth 0
Interpreter and browser resolution	Python 3.11.15 in *.venv*; Chrome present at its standard Windows path

7.1.3 PERFORMANCE AND DESIGN DATA.

Table 7.1-2. Environment Variable Names

Name	Required	Effect
NVIDIA_API_KEY	for LLM/RAG	Embeddings and explanations
NVIDIA_BASE_URL	no	Endpoint override
NVIDIA_MODEL	no	Chat model override
GITHUB_TOKEN	no	Private repos; 5,000 req/h
CHROMA_DB_PATH	no	Persistence directory
REDIS_URL	no	Enables Redis mode
RQ_QUEUE_NAME	no	Default *gnosis_jobs*
JOB_TIMEOUT_SECONDS	no	Default 1,800
JOB_TTL_SECONDS	no	Result expiry
MAX_*_ANALYSIS_FILES	no	Tier thresholds
PORT	worker	Health listener
VITE_API_URL	frontend	API origin at build time

CAUTION

Not executed: *docker compose up*, *python -m src.api.worker*, *npm ci*, *npm run build*, and any analysis of a real repository. Their documented behaviour is described earlier from code, but this handbook makes no claim that they succeed on this machine.

```
START CHECK                                  OPERATOR
1 RUN - pytest, expect 476
2 RUN - run.py --help
  √ START - /health returns ok
3 READ - queue_mode and redis flag
4 STOP - terminate listener
```

OPS

SECTION 7 · SUBSECTION 7.2
TEST AND VERIFICATION PROGRAM

7.2.1 INTRODUCTION.

The suite is broad, offline and fast: 476 cases in roughly twenty seconds. It is strongest on pure transformations and HTTP contracts, and weakest on the seams between stages. That shape is typical of a system built stage by stage: each agent has detailed unit tests, while the orchestration that connects them is covered only indirectly. Read a green run as evidence that the parts behave, not that the whole path has been exercised (figure 7.2-1).

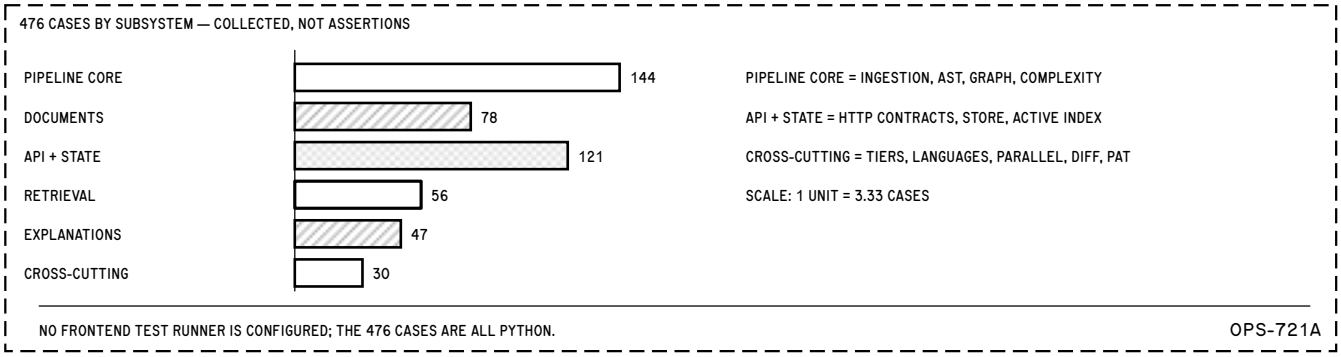


Figure 7.2-1. Case Distribution Across Subsystems

7.2.2 FUNCTIONAL DESCRIPTION.

Unit tests build source strings, parse them and assert on dataclasses; they do not need a repository. API tests import the FastAPI application, replace the pipeline submission with a fake, and drive routes through *TestClient*. Store tests run the same lifecycle twice, once against the memory backend and once against *fakeredis* with WATCH semantics. The active-index tests simulate contention by mutating keys between a read and a write, and cover lost sentinels, external members and interrupted reconciliation.

The resilience tests are the most informative for an operator: partial model failure, missing collection, missing key, terminal-record protection and retry exhaustion all have explicit cases. Multi-language tests cover symbol extraction and complexity for Go, Rust, Java and C++, plus alias and layout resolution. Two cases assert that concurrent parsing and scoring return the same state as sequential execution.

Table 7.2-1. Test Layers and What They Establish

Layer	Cases	What a pass establishes
Ingestion and filters	22	URL forms, extension and directory rules, size ceiling, language labels
Parsing and symbols	34	Python and JS declarations, docstrings, imports and parse-error capture
Graph resolution	38	Internal versus external imports, cycles, order, stats, PageRank fallback
Complexity and risk	50	Branch counts, threshold boundaries, precedence, coupling from graph stats
Retrieval	56	Chunk identity, collection naming, metadata filters, batch embedding
Explanations	47	Selection priority, context budget, prompt content, sanitisation, partial failure
Documents	78	Section presence, order, tables, empty-state safety, heading hierarchy
HTTP and state	121	Status codes, validation, lifecycle, health, duplicate submission, index drift
Cross-cutting	30	Tiers, four extra languages, parallel equivalence, diff, private-repo headers

7.2.3 PERFORMANCE AND DESIGN DATA.

Table 7.2-2. Observed Run and Uncovered Paths

Item	Value
Result	476 passed
Wall time	20.40 s
Files	15
Failures / skips	0 / 0
Network calls	none observed
Frontend tests	none configured
Worker loop	handler only
CLI savers	not end-to-end
Incremental delete path	not covered
Real Chroma persistence	not covered

WARNING

A green suite does not cover the undefined *logger* in incremental chunk deletion, the missing on-disk *agent_context.md*, the method-count defect in the agent context, or the cleared *raw_contents*. Each sits on a seam between tested units.

TEST CHECK	ENGINEER
1 RUN - full suite, record time	
2 READ - first failure, not the count	
√ KNOW - what is not tested	
3 ADD - a seam test before a fix	

OPS

SECTION 8 · SUBSECTION 8.1
KNOWN DEFECTS AND LIMITATIONS

8.1.1 INTRODUCTION.

This register lists defects and limitations confirmed by reading the working tree. Each entry names a file, the symptom, and the class of harm. None was fixed for this handbook. Severity is an engineering judgement, not a project rating: **HIGH** means a lost deliverable, a failed run on a supported path, or a credential exposure; **MEDIUM** means misleading output or an operational trap; **LOW** means dead code, stale text or cosmetic drift (figure 8.1-1).

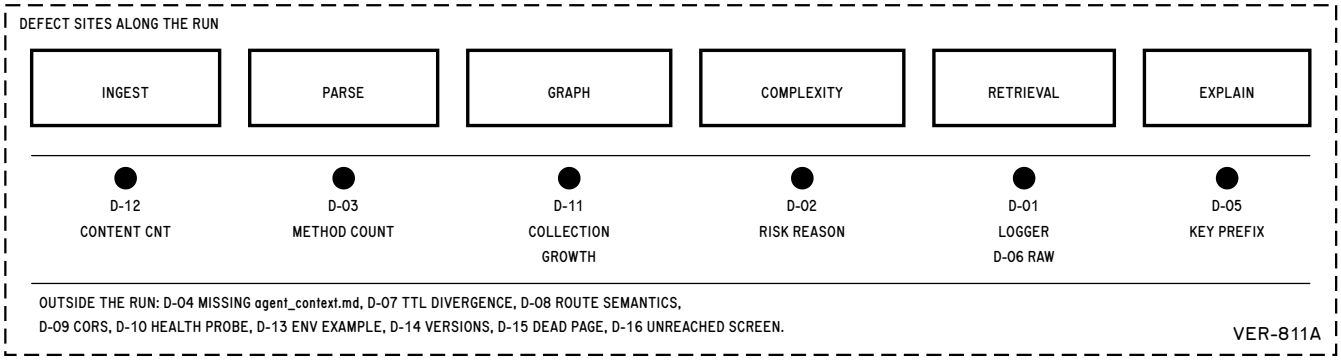


Figure 8.1-1. Where Each Confirmed Defect Enters the Run

8.1.2 DEFECT REGISTER.

Locations are line numbers at the change date. “Latent” means the path is not exercised by the current suite; it does not mean the path is unreachable in production.

Table 8.1-1. Confirmed Defects, Limitations and Their Effects

ID	Location	Severity	Symptom and consequence
D-01	agents/code_rag.py:109	HIGH	*logger* is undefined; a failed stale-chunk delete raises *NameError* and aborts the run instead of logging. Latent: needs incremental mode plus a delete failure.
D-02	agents/complexity_scorer.py:297	MEDIUM	CRITICAL reason prints the complexity value where the function name belongs, e.g. “Function '24' has cyclomatic complexity 24”.
D-03	agents/doc_generator.py:738	MEDIUM	*ClassInfo* exposes *method_names*, but the section reads *methods*; every class is reported with 0 methods.
D-04	run.py:149–157	HIGH	*agent_context_md* is generated but never saved by the CLI; only the API returns it. The advertised deliverable is missing on disk.
D-05	agents/explainability.py:61	HIGH	The first eight characters of the API key are printed to logs. Any log aggregation or screenshot leaks key material.
D-06	agents/code_rag.py:246	MEDIUM	Raw text is cleared before explainability's fallback reads it; Agent 6 must rely on Chroma chunks and silently loses context if retrieval stored none.
D-07	api/job_store.py:345	MEDIUM	RQ results expire after 86,400 s, but job records default to no expiry. The record outlives its result and reports a complete job with nothing to return.
D-08	api/main.py:238–254	MEDIUM	*/api/jobs/{id}* returns HTTP 200 with *{"error":"job not found"}*, while */jobs/{id}* returns 404. Clients must branch on body shape.
D-09	api/main.py:55–59	MEDIUM	CORS allows every origin with credentials enabled. Starlette reflects the caller's origin for credentialed requests, so any site can call the API with cookies if a browser has them.
D-10	api/worker.py:45–56	MEDIUM	The worker health endpoint always answers “ok”; it does not test Redis, queue depth or the pipeline. A live worker can be idle or wedged.
D-11	utils/retriever.py:make_collection_name	MEDIUM	Collections are per job; only *DELETE /jobs/{id}* removes one, best-effort and never called by the client. Disk use grows with every run.
D-12	orchestrator.py:166	LOW	*files_with_content* is computed after retrieval clears raw text, so the manifest always reports 0 despite successful ingestion.
D-13	.env.example:17	LOW	Example model is *llama-3.3-70b-instruct*; code default is *llama-3.2-11b-vision-instruct*. Copying the example changes behaviour.
D-14	main.py:47, App.tsx:8, package.json	LOW	API reports 1.0.0, client 1.1.0, package 1.0.0, change log has two 1.1.0 entries. No single version authority.
D-15	frontend/src/pages/ResultsPage.tsx	LOW	762 lines of an earlier results implementation are compiled by nothing; *react-router-dom* is installed but unused.
D-16	frontend/src/App.tsx:23, 1646	LOW	The *error* screen is declared and rendered but never set; failures appear inside the progress screen instead.

WARNING

D-05 is a credential-handling fault, not a cosmetic log line. The value is truncated, but a prefix narrows a brute-force search and is enough to correlate keys across logs. Do not paste production logs into issues or documents.

DEFECT CHECK — **MAINTAINER**

```
1 REPRODUCE - name the path
2 LOCATE - file and line
  √ CLASS - lost output, wrong output, trap
3 TEST - add a seam case first
4 FIX - one defect per change
```

SECTION 8 · SUBSECTION 8.2
SOURCE VERIFICATION AND OPEN ITEMS

8.2.1 INTRODUCTION.

This appendix closes the evidence trail. Every substantive claim in the handbook can be traced to a file, a test result or a named absence. No live deployment, remote repository, paid model call or production database was inspected. Where the tree cannot answer a question, the question is recorded rather than guessed (figure 8.2-1).

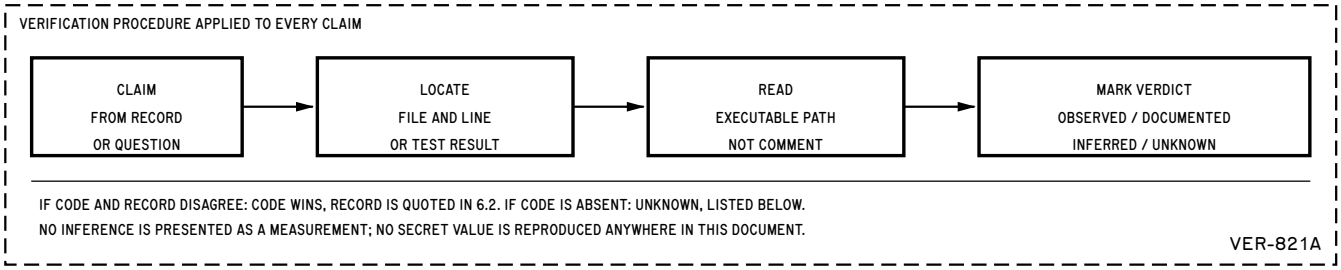


Figure 8.2-1. Claim-to-Evidence Procedure Used Throughout

8.2.2 SOURCE-VERIFICATION INDEX.

Line ranges refer to the working tree at the change date. They are starting points, not exhaustive citations.

Table 8.2-1. Principal Claims and Their Evidence

Claim	Evidence	Mark
Seven stages; middle three concurrent	src/orchestrator.py:85–145; src/api/pipeline_runner.py:86–122	OBSERVED
State is the only cross-agent contract	src/state.py:8–61; agent module imports	OBSERVED
Tier thresholds 300 / 1,000 / 3,000	src/agents/ingestion.py:9–73	OBSERVED
18 extensions, 24 excluded dirs, 150 KB	src/utils/filters.py:7–94	OBSERVED
Embedding model, 2,048 dimensions, batch 50	src/utils/embedder.py:25–28	OBSERVED
Chat model, two attempts, 120 s, 2 s pause	src/utils/nvidia_client.py:40–51, 245	OBSERVED
Risk thresholds 21/11, 10/6, 8/4, 0.80/0.60	src/agents/complexity_scorer.py:31–52	OBSERVED
Redis keys, sentinel, five WATCH attempts	src/api/job_store.py:31–35, 127–129	OBSERVED
Record TTL defaults to none; RQ TTL 86,400	src/api/job_store.py:344–351; src/api/queue.py	OBSERVED
Dual route families and status codes	src/api/main.py:155–343	OBSERVED
Client polls 1,500 ms, six tabs, no router	frontend/src/App.tsx:559–625, 921, 1601	OBSERVED
476 tests pass in 20.40 s	*pytest* and *--collect-only* runs, 27 October 2026	OBSERVED
Local API starts and /health answers	uvicorn on port 8011; response recorded on 7.1-1	OBSERVED
Azure deployment is live and current	Workflows exist; no deployment or run was inspected	UNKNOWN
Model and embedding services are available	No network call was made with a key	UNKNOWN

8.2.3 Deliberate Simplifications.

Figures are redrawn as single-colour line art; no product screenshots are included. The catalogue gives non-empty line counts and one-line responsibilities, not a function-by-function inventory. Pipeline examples are structural: no real repository was analysed, so no timing, cost or output-quality measurement is offered. Secret values and the contents of *.env* are excluded; only variable names appear. Commands not listed on 7.1-1 are described from code and marked as unexecuted.

8.2.4 Open Items.

1. Which version string is authoritative: API 1.0.0, client 1.1.0, package 1.0.0, or the change log's two 1.1.0 entries?
2. Is *Decisions.md* canonical? It is gitignored, yet *agent.md* depends on its rationale and identifiers.
3. Should per-job Chroma collections be deleted, retained, or consolidated per repository? The tree has an on-demand delete endpoint but no retention rule.
4. Does production run Redis and a worker, or the local thread pool? The README describes Azure; no live check was performed.
5. Is *pages/ResultsPage.tsx* awaiting deletion or revival? It is compiled by nothing and duplicates live code.
6. Are the tier environment overrides used in deployment, or are the defaults the effective policy?
7. Is *agent_context.md* expected on disk from the CLI, as the README implies, or only through the API?

END OF VOLUME — 27 SHEETS, SECTIONS 1–8 · 476 TESTS OBSERVED PASSING · 16 DEFECTS RECORDED · 7 OPEN ITEMS · NO SECRETS REPRODUCED · NO LIVE SYSTEM ASSERTED